

Universidad Mariano Gálvez de Guatemala

Facultad de ciencias de la información

Dirección de licenciaturas

Ingeniería en sistemas

Bases de datos 2

Ing. Edgar René Ornelis Hoil



Proyecto final banco distribuido

Brandon Isaías Ampérez Aldana 0909-21-4628

Puerto Barrios, Izabal 6 de junio de 2026

Índice

introducción.....	4
Sistema GBA CREDOMATIC	5
Objetivos del proyecto.....	6
1. Objetivo general	6
2. Objetivos específicos.....	6
Arquitectura del sistema	7
Componentes principales	7
1. Cliente web:	7
2. Frontend:.....	7
3. Backend:	7
4. Base de datos:	7
5. Control de versiones:	7
Flujo general de funcionamiento.....	8
Diseño de base de datos	8
Las tablas principales del sistema.....	8
Relaciones principales entre tablas.....	10
Scripts de las tablas del sistema.	11
Diagrama ER.	23
Diccionario de datos	24
Transacciones y propiedades ACID	29
• Atomicidad	29
• Consistencia	29
• Aislamiento	29
• Durabilidad.....	30
Operaciones transaccionales implementadas	30
Control de concurrencia.....	31
Bloqueos con FOR UPDATE	32
Prevención y manejo de deadlocks	33

Tiempo máximo de espera por bloqueos	34
Triggers de auditoría	35
Scripts de Triggers utilizados.	36
Respaldo y recuperación	55
Respaldo base inicial	56
Respaldo continuo lógico	56
Recuperación de datos.....	56
Scripts de respaldo.....	56
Scripts de recuperación.	84
Optimización de consultas.....	133
Scripts de consultas	133
Análisis comparativo de consultas optimizadas.....	144
Seguridad del sistema	148
Control de acceso por roles	148
Manual de usuario	150
Acceso al sistema	150
Rol administrador	152
Rol cajero	156
Rol auditor.....	159
Cierre de sesión.....	162
Reporte de pruebas	163
Conclusiones	167
Anexos	168
Capturas de pantalla del Sistema GBA CREDOMATIC	168
Capturas de triggers ejecutados.....	178
Capturas de respaldo y recuperación	179
Capturas de consultas optimizadas	180
Enlace al repositorio del proyecto	181

introducción.

El desarrollo de sistemas bancarios requiere una estructura tecnológica capaz de manejar información sensible, operaciones financieras y registros transaccionales de manera segura, ordenada y confiable. El sistema GBA CREDOMATIC fue diseñado como una solución web orientada a la administración de clientes, cuentas bancarias, transacciones, pagos de servicios, usuarios internos, auditoría y reportes administrativos.

La solución fue construida aplicando principios fundamentales de bases de datos relacionales, especialmente en el diseño de tablas, relaciones, restricciones, integridad referencial, manejo de transacciones, control de concurrencia, respaldo, recuperación, auditoría y optimización de consultas. Estos elementos permiten que el sistema no solo almacene información, sino que también mantenga coherencia en las operaciones realizadas y facilite la trazabilidad de los procesos ejecutados por cada usuario.

El sistema utiliza PostgreSQL como motor de base de datos, debido a su capacidad para manejar transacciones, relaciones, restricciones, funciones, triggers, respaldos y consultas optimizadas. La aplicación fue desarrollada con Python y Flask, permitiendo construir una arquitectura cliente-servidor en la que los usuarios interactúan desde un navegador web y el servidor procesa las operaciones hacia la base de datos.

GBA CREDOMATIC cuenta con distintos roles de usuario, entre ellos administrador, cajero y auditor. Cada rol posee funciones específicas para mantener una separación adecuada de responsabilidades. El administrador gestiona usuarios, clientes y cuentas; el cajero realiza operaciones bancarias; y el auditor revisa los eventos registrados dentro del sistema. Esta división fortalece el control interno y permite simular un entorno bancario más realista.

Sistema GBA CREDOMATIC

El sistema GBA CREDOMATIC es una aplicación bancaria web diseñada para administrar procesos básicos de una entidad financiera. Su funcionamiento se centra en la gestión de clientes, apertura de cuentas, procesamiento de transacciones, pagos de servicios, administración de usuarios, auditoría y generación de reportes.

El sistema cuenta con tres roles principales: administrador, cajero y auditor. Esta separación permite controlar las acciones que cada usuario puede realizar dentro de la plataforma, evitando que todos los usuarios tengan acceso completo a las mismas funciones.

El administrador tiene acceso a las funciones de gestión interna del sistema. Puede crear usuarios, consultar información general, activar o inactivar usuarios, clientes y cuentas, revisar reportes y visualizar estadísticas relacionadas con el funcionamiento del banco.

El cajero se encarga de las operaciones bancarias directas. Puede registrar clientes, crear cuentas, realizar depósitos, procesar retiros, ejecutar transferencias entre cuentas, registrar pagos de servicios y anular transacciones cuando sea necesario.

El auditor tiene acceso a la revisión de eventos del sistema. Su función principal es consultar la bitácora de auditoría, verificar las operaciones realizadas y revisar la trazabilidad de las acciones ejecutadas por los usuarios internos.

El sistema también incluye validaciones para proteger la integridad de los datos. Por ejemplo, evita operaciones con montos inválidos, controla fondos insuficientes en retiros y transferencias, registra acciones importantes en auditoría y restringe funciones según el rol del usuario autenticado.

Objetivos del proyecto

1. Objetivo general

Desarrollar un sistema bancario funcional que permita administrar usuarios, clientes, cuentas, transacciones, pagos y registros de auditoría, aplicando conceptos fundamentales de bases de datos relacionales, seguridad, control transaccional, respaldo, recuperación y optimización de consultas.

2. Objetivos específicos

- Diseñar una base de datos relacional en PostgreSQL que represente correctamente la estructura de información de un sistema bancario.
- Implementar una aplicación web con Python y Flask que permita interactuar con los datos almacenados en PostgreSQL.
- Gestionar usuarios internos mediante roles diferenciados, permitiendo el acceso controlado a los módulos de administrador, cajero y auditor.
- Registrar clientes y cuentas bancarias, manteniendo relaciones correctas entre cada cliente, sus cuentas y los tipos de cuenta disponibles.
- Procesar operaciones bancarias como depósitos, retiros, transferencias, pagos de servicios y anulaciones, respetando la integridad de los saldos y registros.
- Aplicar propiedades ACID en las operaciones transaccionales para garantizar consistencia, atomicidad, aislamiento y durabilidad.
- Implementar triggers de auditoría que registren automáticamente eventos importantes dentro del sistema.
- Crear mecanismos de respaldo y recuperación que permitan proteger y restaurar información crítica.
- Optimizar consultas mediante índices en columnas utilizadas frecuentemente en búsquedas, uniones y ordenamientos.
- Documentar el funcionamiento técnico y operativo del sistema para facilitar su comprensión, evaluación y presentación final.

Arquitectura del sistema

El sistema utiliza una arquitectura cliente-servidor. El cliente accede desde un navegador web, mientras que el servidor ejecuta la lógica de negocio mediante Flask y se comunica con PostgreSQL para almacenar, consultar y modificar la información.

Componentes principales

1. Cliente web:

Representa el navegador utilizado por los usuarios para acceder al sistema. Desde esta capa se visualizan los formularios, paneles, tablas, reportes y mensajes del sistema.

2. Frontend:

Está desarrollado con HTML, CSS, JavaScript y plantillas Jinja2. Esta capa presenta la información al usuario y permite la interacción con los módulos del sistema.

3. Backend:

Está desarrollado con Python y Flask. Se encarga de procesar solicitudes, validar datos, controlar sesiones, aplicar restricciones por rol y ejecutar operaciones contra la base de datos.

4. Base de datos:

PostgreSQL almacena la información principal del sistema, incluyendo usuarios, roles, clientes, cuentas, transacciones, pagos, servicios, auditoría y solicitudes de contacto.

5. Control de versiones:

El proyecto se administra mediante Git, permitiendo registrar cambios, trabajar con compañeros y mantener un historial de commits. El repositorio puede alojarse en GitHub.

Flujo general de funcionamiento

El usuario accede al sistema desde el navegador e ingresa sus credenciales. Flask valida la información contra la tabla de usuarios en PostgreSQL y verifica el estado del usuario. Si las credenciales son correctas, el sistema identifica el rol y redirige al panel correspondiente.

Cuando el usuario realiza una operación, Flask valida los datos recibidos y ejecuta las consultas necesarias en PostgreSQL. En operaciones importantes, como creación de usuarios, registro de clientes, apertura de cuentas o transacciones bancarias, PostgreSQL ejecuta triggers que registran automáticamente los eventos en la tabla de auditoría.

Finalmente, el sistema devuelve una respuesta al usuario mediante mensajes, tablas, reportes o redirecciones dentro de la interfaz web.

Diseño de base de datos

La base de datos fue diseñada bajo un modelo relacional, utilizando tablas conectadas mediante llaves primarias y llaves foráneas. Esta estructura permite mantener integridad en los datos y representar correctamente las relaciones entre usuarios, clientes, cuentas y operaciones bancarias.

Las tablas principales del sistema.

- **roles**

La tabla roles almacena los tipos de usuarios internos del sistema. Permite diferenciar los permisos y responsabilidades de cada usuario, como administrador, cajero o auditor.

- **usuarios**

La tabla usuarios almacena las credenciales, datos personales, estado y rol asignado a cada usuario interno. Esta tabla permite controlar el acceso al sistema y definir qué funciones puede utilizar cada usuario según su rol.

- **clientes**

La tabla clientes contiene la información principal de los clientes del banco, como DPI, nombres, apellidos, teléfono, correo, dirección y estado. Esta tabla sirve como base para asociar a cada cliente con una o varias cuentas bancarias.

- **cuentas**

La tabla cuentas registra las cuentas bancarias asociadas a los clientes. Contiene información como número de cuenta, tipo de cuenta, saldo, estado y fecha de apertura. Es una de las tablas principales del sistema, ya que sobre ella se realizan operaciones como depósitos, retiros, transferencias y pagos.

- **tipos_cuenta**

La tabla tipos_cuenta funciona como catálogo para clasificar las cuentas bancarias. Permite identificar si una cuenta corresponde, por ejemplo, a una cuenta de ahorro, monetaria u otro tipo definido por el sistema.

- **transacciones**

La tabla transacciones almacena las operaciones bancarias realizadas dentro del sistema. Registra depósitos, retiros, transferencias, pagos de servicio y anulaciones. Esta tabla permite mantener el historial de movimientos financieros y controlar los cambios realizados en los saldos de las cuentas.

- **pagos**

La tabla pagos registra los pagos de servicios realizados desde cuentas bancarias. Guarda información como el código del pago, cuenta asociada, servicio pagado, referencia, monto, estado y fecha del pago.

- **servicios**

La tabla servicios funciona como catálogo de servicios disponibles para pago dentro del sistema. Puede incluir servicios como energía eléctrica, agua, internet u otros servicios definidos para las operaciones bancarias.

- **auditoria**

La tabla auditoria almacena los eventos importantes realizados dentro del sistema. Permite revisar qué usuario ejecutó una acción, en qué módulo ocurrió, qué operación se realizó, cuál fue la descripción del evento y en qué fecha sucedió. Esta tabla es fundamental para la trazabilidad y control interno.

Relaciones principales entre tablas

1. La estructura de la base de datos mantiene relaciones entre las tablas para asegurar la integridad de la información y evitar registros aislados o inconsistentes.
2. Un rol puede estar asignado a varios usuarios, pero cada usuario pertenece a un rol específico.
3. Un cliente puede tener una o varias cuentas bancarias, mientras que cada cuenta pertenece únicamente a un cliente.
4. Una cuenta pertenece a un tipo de cuenta, lo cual permite clasificarla según su naturaleza o uso.
5. Una transacción puede estar asociada a una cuenta origen, a una cuenta destino o a ambas, dependiendo del tipo de operación realizada.
6. Un pago se relaciona con una cuenta bancaria y con un servicio específico.
7. Un registro de auditoría puede estar asociado a un usuario interno del sistema, permitiendo identificar quién realizó una acción determinada.

Scripts de las tablas del sistema.

```
-- =====  
  
-- BASE DE DATOS: Proyecto_Banco_Distribuido  
  
-- SISTEMA: GBA CREDOMATIC  
  
-- =====
```

```
-- =====  
  
-- 1. TABLA DE ROLES  
  
-- =====
```

```
CREATE TABLE IF NOT EXISTS roles (  
    id_rol SERIAL PRIMARY KEY,  
    nombre_rol VARCHAR(50) NOT NULL UNIQUE,  
    descripcion TEXT,  
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO',  
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- =====  
  
-- 2. TABLA DE USUARIOS INTERNOS  
  
-- =====
```

```
CREATE TABLE IF NOT EXISTS usuarios (
```

```

id_usuario SERIAL PRIMARY KEY,

id_rol INTEGER NOT NULL,

nombre_completo VARCHAR(150) NOT NULL,

usuario VARCHAR(80) NOT NULL UNIQUE,

correo VARCHAR(120) NOT NULL UNIQUE,

clave VARCHAR(255) NOT NULL,

estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO',

fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


CONSTRAINT fk_usuario_rol

    FOREIGN KEY (id_rol)

    REFERENCES roles(id_rol)

    ON UPDATE CASCADE

    ON DELETE RESTRICT

);


-- =====

-- 3. TABLA DE CLIENTES

-- =====


CREATE TABLE IF NOT EXISTS clientes (

    id_cliente SERIAL PRIMARY KEY,

    dpi VARCHAR(20) NOT NULL UNIQUE,

```

```

nombres VARCHAR(100) NOT NULL,
apellidos VARCHAR(100) NOT NULL,
telefono VARCHAR(20),
correo VARCHAR(120),
direccion TEXT,
estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO',
fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

-- =====
-- 4. TABLA DE TIPOS DE CUENTA
-- =====

```

```

CREATE TABLE IF NOT EXISTS tipos_cuenta (
    id_tipo_cuenta SERIAL PRIMARY KEY,
    nombre_tipo VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO'
);

```

```

-- =====
-- 5. TABLA DE CUENTAS BANCARIAS
-- =====

```

```

CREATE TABLE IF NOT EXISTS cuentas (

    id_cuenta SERIAL PRIMARY KEY,

    id_cliente INTEGER NOT NULL,

    id_tipo_cuenta INTEGER NOT NULL,

    numero_cuenta VARCHAR(30) NOT NULL UNIQUE,

    saldo NUMERIC(12,2) NOT NULL DEFAULT 0.00,

    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA',

    fecha_apertura TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


    CONSTRAINT fk_cuenta_cliente

        FOREIGN KEY (id_cliente)

        REFERENCES clientes(id_cliente)

        ON UPDATE CASCADE

        ON DELETE RESTRICT,


    CONSTRAINT fk_cuenta_tipo

        FOREIGN KEY (id_tipo_cuenta)

        REFERENCES tipos_cuenta(id_tipo_cuenta)

        ON UPDATE CASCADE

        ON DELETE RESTRICT,


    CONSTRAINT chk_saldo_no_negativo

```

```

        CHECK (saldo >= 0)

);

-- =====

-- 6. TABLA DE TRANSACCIONES

-- =====

CREATE TABLE IF NOT EXISTS transacciones (

    id_transaccion SERIAL PRIMARY KEY,

    codigo_transaccion VARCHAR(40) NOT NULL UNIQUE,

    tipo_transaccion VARCHAR(30) NOT NULL,

    id_cuenta_origen INTEGER,

    id_cuenta_destino INTEGER,

    monto NUMERIC(12,2) NOT NULL,

    estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA',

    descripcion TEXT,

    fecha_transaccion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_transaccion_origen

        FOREIGN KEY (id_cuenta_origen)

        REFERENCES cuentas(id_cuenta)

        ON UPDATE CASCADE

        ON DELETE RESTRICT,

```

CONSTRAINT fk_transaccion_destino

FOREIGN KEY (id_cuenta_destino)

REFERENCES cuentas(id_cuenta)

ON UPDATE CASCADE

ON DELETE RESTRICT,

CONSTRAINT chk_monto_transaccion

CHECK (monto > 0),

CONSTRAINT chk_tipo_transaccion

CHECK (tipo_transaccion IN ('DEPOSITO', 'RETIRO', 'TRANSFERENCIA',
'PAGO_SERVICIO'))

);

-- =====

-- 7. TABLA DE SERVICIOS

-- =====

CREATE TABLE IF NOT EXISTS servicios (

id_servicio SERIAL PRIMARY KEY,

nombre_servicio VARCHAR(80) NOT NULL UNIQUE,

descripcion TEXT,


```

    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO'

);

-- =====

-- 8. TABLA DE PAGOS DE SERVICIOS

-- =====

CREATE TABLE IF NOT EXISTS pagos (

    id_pago SERIAL PRIMARY KEY,

    codigo_pago VARCHAR(40) NOT NULL UNIQUE,

    id_cuenta INTEGER NOT NULL,

    id_servicio INTEGER NOT NULL,

    referencia VARCHAR(100) NOT NULL,

    monto NUMERIC(12,2) NOT NULL,

    estado VARCHAR(20) NOT NULL DEFAULT 'PAGADO',

    fecha_pago TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_pago_cuenta

        FOREIGN KEY (id_cuenta)

        REFERENCES cuentas(id_cuenta)

        ON UPDATE CASCADE

        ON DELETE RESTRICT,

```

```

CONSTRAINT fk_pago_servicio

    FOREIGN KEY (id_servicio)

    REFERENCES servicios(id_servicio)

    ON UPDATE CASCADE

    ON DELETE RESTRICT,


CONSTRAINT chk_monto_pago

    CHECK (monto > 0)

);


-- =====
-- 9. TABLA DE AUDITORÍA
-- =====


CREATE TABLE IF NOT EXISTS auditoria (

    id_auditoria SERIAL PRIMARY KEY,

    id_usuario INTEGER,

    modulo VARCHAR(80) NOT NULL,

    operacion VARCHAR(80) NOT NULL,

    descripcion TEXT NOT NULL,

    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


CONSTRAINT fk_auditoria_usuario

```

```

        FOREIGN KEY (id_usuario)
        REFERENCES usuarios(id_usuario)
        ON UPDATE CASCADE
        ON DELETE SET NULL
    );

-- =====

-- 10. TABLA DE SOLICITUDES DE CONTACTO PÚBLICAS

-- =====

CREATE TABLE IF NOT EXISTS solicitudes_contacto (
    id_solicitud SERIAL PRIMARY KEY,
    nombre_completo VARCHAR(150) NOT NULL,
    correo VARCHAR(120) NOT NULL,
    mensaje TEXT NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
    fecha_solicitud TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====

-- 11. INSERCIÓN DE ROLES BASE

-- =====

```

```

INSERT INTO roles (nombre_rol, descripcion)

VALUES

('ADMINISTRADOR', 'Gestiona usuarios, seguridad, reportes, auditoría y configuración
del sistema.'),

('CAJERO', 'Registra clientes, crea cuentas, procesa transacciones y pagos.'),

('AUDITOR', 'Consulta información, revisa eventos y supervisa operaciones del sistema.')

ON CONFLICT (nombre_rol) DO NOTHING;

```

```

-- =====

-- 12. INSERCIÓN DE TIPOS DE CUENTA

-- =====

```

```

INSERT INTO tipos_cuenta (nombre_tipo, descripcion)

VALUES

('AHORRO', 'Cuenta de ahorro para clientes individuales.'),

('MONETARIA', 'Cuenta monetaria para operaciones frecuentes.'),

('EMPRESARIAL', 'Cuenta bancaria para empresas y negocios.')

ON CONFLICT (nombre_tipo) DO NOTHING;

```

```

-- =====

-- 13. INSERCIÓN DE SERVICIOS BASE

-- =====

```

```

INSERT INTO servicios (nombre_servicio, descripcion)
VALUES
('ENERGIA ELECTRICA', 'Pago de servicio de energía eléctrica.'),
('AGUA POTABLE', 'Pago de servicio de agua potable.'),
('INTERNET', 'Pago de servicio de internet residencial o empresarial.'),
('TELEFONIA', 'Pago de servicio de telefonía.')
ON CONFLICT (nombre_servicio) DO NOTHING;

-- =====

-- 14. ÍNDICES

-- =====

CREATE INDEX IF NOT EXISTS idx_usuarios_usuario
ON usuarios(usuario);

CREATE INDEX IF NOT EXISTS idx_usuarios_correo
ON usuarios(correo);

CREATE INDEX IF NOT EXISTS idx_clientes_dpi
ON clientes(dpi);

CREATE INDEX IF NOT EXISTS idx_cuentas_numero
ON cuentas(numero_cuenta);

```

```
CREATE INDEX IF NOT EXISTS idx_transacciones_fecha  
ON transacciones(fecha_transaccion);
```

```
CREATE INDEX IF NOT EXISTS idx_auditoria_fecha  
ON auditoria(fecha);
```

```
CREATE INDEX IF NOT EXISTS idx_auditoria_usuario  
ON auditoria(id_usuario);
```

Diagrama ER.

Diagrama ER GBA CREDOMATIC



Diccionario de datos

Tabla roles

Campo	Tipo de dato	Descripción
id_rol	SERIAL / INTEGER	Identificador único del rol.
nombre_rol	VARCHAR	Nombre del rol, por ejemplo ADMINISTRADOR, CAJERO o AUDITOR.
descripcion	TEXT / VARCHAR	Descripción general del rol.
estado	VARCHAR	Estado del rol, normalmente ACTIVO o INACTIVO.
fecha_registro	TIMESTAMP	Fecha en que fue registrado el rol.

Tabla usuarios

Campo	Tipo de dato	Descripción
id_usuario	SERIAL / INTEGER	Identificador único del usuario.
id_rol	INTEGER	Llave foránea que relaciona el usuario con un rol.
nombre_completo	VARCHAR	Nombre completo del usuario interno.
usuario	VARCHAR	Nombre de usuario utilizado para iniciar sesión.
correo	VARCHAR	Correo electrónico del usuario.
clave	VARCHAR	Contraseña cifrada del usuario.

estado	VARCHAR	Estado del usuario, como ACTIVO o INACTIVO.
fecha_registro	TIMESTAMP	Fecha de creación del usuario.

Tabla clientes

Campo	Tipo de dato	Descripción
id_cliente	SERIAL / INTEGER	Identificador único del cliente.
dpi	VARCHAR	Documento Personal de Identificación del cliente.
nombres	VARCHAR	Nombres del cliente.
apellidos	VARCHAR	Apellidos del cliente.
telefono	VARCHAR	Número telefónico del cliente.
correo	VARCHAR	Correo electrónico del cliente.
direccion	TEXT / VARCHAR	Dirección del cliente.
estado	VARCHAR	Estado del cliente, como ACTIVO o INACTIVO.
fecha_registro	TIMESTAMP	Fecha en que se registró el cliente.

Tabla cuentas

Campo	Tipo de dato	Descripción
id_cuenta	SERIAL / INTEGER	Identificador único de la cuenta.

id_cliente	INTEGER	Llave foránea que relaciona la cuenta con un cliente.
id_tipo_cuenta	INTEGER	Llave foránea que relaciona la cuenta con el tipo de cuenta.
numero_cuenta	VARCHAR	Número único de cuenta bancaria.
saldo	NUMERIC	Saldo disponible en la cuenta.
estado	VARCHAR	Estado de la cuenta, como ACTIVA o INACTIVA.
fecha_apertura	TIMESTAMP	Fecha en que fue creada la cuenta.

Tabla tipos_cuenta

Campo	Tipo de dato	Descripción
id_tipo_cuenta	SERIAL / INTEGER	Identificador único del tipo de cuenta.
nombre_tipo	VARCHAR	Nombre del tipo de cuenta.
descripcion	TEXT / VARCHAR	Descripción del tipo de cuenta.
estado	VARCHAR	Estado del tipo de cuenta.

Tabla transacciones

Campo	Tipo de dato	Descripción
id_transaccion	SERIAL / INTEGER	Identificador único de la transacción.
codigo_transaccion	VARCHAR	Código único generado para la transacción.

tipo_transaccion	VARCHAR	Tipo de operación: DEPOSITO, RETIRO, TRANSFERENCIA o PAGO_SERVICIO.
id_cuenta_origen	INTEGER	Cuenta desde donde sale el dinero, si aplica.
id_cuenta_destino	INTEGER	Cuenta hacia donde entra el dinero, si aplica.
monto	NUMERIC	Monto de la operación.
estado	VARCHAR	Estado de la transacción, como COMPLETADA o ANULADA.
descripcion	TEXT	Descripción de la operación realizada.
fecha_transaccion	TIMESTAMP	Fecha y hora de la transacción.

Tabla pagos

Campo	Tipo de dato	Descripción
id_pago	SERIAL / INTEGER	Identificador único del pago.
codigo_pago	VARCHAR	Código único del pago realizado.
id_cuenta	INTEGER	Cuenta desde donde se realiza el pago.
id_servicio	INTEGER	Servicio que se está pagando.
referencia	VARCHAR	Número o código de referencia del pago.
monto	NUMERIC	Monto pagado.
estado	VARCHAR	Estado del pago, como PAGADO.
fecha_pago	TIMESTAMP	Fecha y hora en que se realizó el pago.

Tabla servicios

Campo	Tipo de dato	Descripción
id_servicio	SERIAL / INTEGER	Identificador único del servicio.
nombre_servicio	VARCHAR	Nombre del servicio disponible para pago.
descripcion	TEXT / VARCHAR	Descripción del servicio.
estado	VARCHAR	Estado del servicio.

Tabla auditoria

Campo	Tipo de dato	Descripción
id_auditoria	SERIAL / INTEGER	Identificador único del evento de auditoría.
id_usuario	INTEGER	Usuario relacionado con la acción realizada.
modulo	VARCHAR	Módulo donde ocurrió la acción.
operacion	VARCHAR	Operación realizada.
descripcion	TEXT	Detalle del evento registrado.
fecha	TIMESTAMP	Fecha y hora del evento.

Transacciones y propiedades ACID

Las transacciones representan una parte fundamental del sistema bancario, ya que permiten ejecutar operaciones financieras de forma segura y controlada. En GBA CREDOMATIC se manejan operaciones como depósitos, retiros, transferencias, pagos de servicios y anulaciones. Cada una de estas operaciones debe mantener la consistencia de los saldos y registrar correctamente la información en la base de datos.

Una transacción bancaria no puede ejecutarse de manera incompleta. Por ejemplo, en una transferencia, el sistema debe descontar dinero de la cuenta origen y acreditar el monto a la cuenta destino. Si una de las dos acciones falla, la operación completa debe revertirse para evitar inconsistencias en los saldos.

- **Atomicidad**

La atomicidad garantiza que una operación se ejecute completamente o no se ejecute. En el sistema, esto se aplica principalmente en transferencias, retiros, depósitos, pagos y anulaciones.

Por ejemplo, si durante una transferencia se descuenta dinero de la cuenta origen, pero ocurre un error antes de acreditar el dinero a la cuenta destino, la operación debe cancelarse y los datos deben regresar a su estado anterior.

En Flask, esto se controla mediante el uso de `commit()` cuando la operación finaliza correctamente y `rollback()` cuando ocurre un error.

- **Consistencia**

La consistencia asegura que la base de datos pase de un estado válido a otro estado válido. En el sistema, esto significa que los saldos no deben quedar negativos, las cuentas deben existir antes de registrar transacciones y las relaciones entre tablas deben mantenerse correctamente.

Por ejemplo, una transacción no debe asociarse a una cuenta inexistente. Además, un retiro no debe permitirse si la cuenta no tiene fondos suficientes.

- **Aislamiento**

El aislamiento permite que una operación no afecte incorrectamente a otra cuando varias acciones se realizan al mismo tiempo. En un sistema bancario, esto es importante porque dos usuarios podrían intentar operar sobre la misma cuenta.

Para controlar este comportamiento, el sistema utiliza consultas con bloqueo como `FOR UPDATE` en operaciones críticas. Esto permite bloquear temporalmente el registro de una cuenta mientras se verifica el saldo y se actualiza la información.

- **Durabilidad**

La durabilidad garantiza que, una vez confirmada una operación, los datos permanezcan guardados en la base de datos. En el sistema, cuando una transacción se confirma mediante `commit()`, los cambios quedan almacenados en PostgreSQL.

Esto significa que un depósito, retiro, transferencia o pago confirmado no debe perderse después de finalizar la operación. Además, la durabilidad se complementa con los mecanismos de respaldo y recuperación implementados en la base de datos.

Operaciones transaccionales implementadas

El sistema implementa varias operaciones transaccionales importantes:

- Depósito: incrementa el saldo de una cuenta destino y registra la operación.
- Retiro: valida fondos suficientes, descuenta el monto de la cuenta origen y registra la operación.
- Transferencia: descuenta el monto de una cuenta origen y acredita el dinero a una cuenta destino.
- Pago de servicio: descuenta el monto de una cuenta y registra el pago asociado a un servicio.
- Anulación de transacción: revierte una operación completada cuando es permitido por el sistema.
- Estas operaciones demuestran el uso de transacciones bancarias controladas, manteniendo integridad, trazabilidad y consistencia en la base de datos.

Control de concurrencia

El control de concurrencia permite administrar correctamente las operaciones que se ejecutan al mismo tiempo dentro de la base de datos. En un sistema bancario, este aspecto es importante porque varios usuarios pueden intentar consultar o modificar información simultáneamente.

Un caso común sería que dos cajeros intenten realizar operaciones sobre la misma cuenta al mismo tiempo. Si el sistema no controla correctamente esta situación, podrían generarse errores en el saldo o registros duplicados.

Para evitar este problema, el sistema utiliza mecanismos de bloqueo en operaciones críticas. Cuando se realiza un retiro, una transferencia o una anulación, la cuenta afectada se bloquea temporalmente mientras se valida el saldo y se actualiza la información.

El uso de FOR UPDATE permite que PostgreSQL bloquee el registro seleccionado hasta que la operación termine. De esta manera, otro proceso no puede modificar ese mismo registro al mismo tiempo.

Este mecanismo ayuda a mantener la integridad de los saldos y evita inconsistencias en operaciones simultáneas.

Ejemplo de control de concurrencia

En una operación de retiro, el sistema primero consulta el saldo de la cuenta utilizando bloqueo. Luego valida si el saldo es suficiente. Si la validación es correcta, descuenta el monto y registra la transacción.

Si otro usuario intenta modificar la misma cuenta en ese momento, deberá esperar hasta que la primera operación finalice. Esto evita que dos operaciones usen el mismo saldo disponible al mismo tiempo.

Importancia en el sistema

El control de concurrencia permite que las operaciones bancarias sean más seguras y confiables. Además, evita errores como saldos negativos, transacciones duplicadas o registros inconsistentes.

Bloqueos con FOR UPDATE

El sistema utiliza la instrucción FOR UPDATE en operaciones donde se modifica información sensible, principalmente saldos de cuentas bancarias. Esta instrucción permite bloquear temporalmente un registro mientras se ejecuta una transacción.

Cuando una cuenta se encuentra bloqueada por una operación, otro proceso no puede modificarla hasta que la transacción actual finalice. Esto evita que dos operaciones alteren el mismo saldo al mismo tiempo.

```
SELECT id_cuenta, saldo
```

```
FROM cuentas
```

```
WHERE id_cuenta = %s
```

```
FOR UPDATE
```

Este mecanismo se aplica en procesos como depósitos, retiros, transferencias, pagos de servicios y anulaciones. Primero se consulta y bloquea la cuenta, luego se valida la información necesaria y finalmente se actualiza el saldo correspondiente.

Al finalizar la operación, PostgreSQL libera el bloqueo después de confirmar los cambios con `commit()` o revertirlos con `rollback()`.

Prevención y manejo de deadlocks

Un deadlock ocurre cuando dos o más operaciones quedan esperando entre sí porque cada una mantiene bloqueado un recurso que la otra necesita. En sistemas bancarios, este riesgo puede presentarse principalmente en transferencias, debido a que una misma operación puede involucrar dos cuentas.

Para reducir este riesgo, el sistema bloquea las cuentas involucradas en una transferencia siguiendo un orden fijo por `id_cuenta`. De esta forma, las cuentas siempre se bloquean en el mismo orden, independientemente de cuál sea la cuenta origen o la cuenta destino.

```
SELECT id_cuenta, saldo  
  
FROM cuentas  
  
WHERE id_cuenta IN (%s, %s)  
  
ORDER BY id_cuenta  
  
FOR UPDATE
```

El uso de `ORDER BY id_cuenta` permite que PostgreSQL bloquee primero la cuenta con menor identificador y luego la siguiente. Esta estrategia evita que dos transferencias simultáneas bloqueen las mismas cuentas en orden contrario.

Además, si PostgreSQL detecta una situación de deadlock, la aplicación captura el error correspondiente, revierte la operación y evita que los datos queden en un estado inconsistente.

```
except errors.DeadlockDetected:
```

```
    conexion.rollback()
```

```
flash("La operación no pudo completarse por un conflicto de concurrencia. Intente nuevamente.", "danger")
```

Este manejo permite mantener la estabilidad del sistema y proteger la información financiera cuando ocurre un conflicto de concurrencia.

Tiempo máximo de espera por bloqueos

El sistema también define tiempos máximos de espera para evitar que una operación permanezca detenida durante demasiado tiempo cuando intenta acceder a un registro bloqueado.

Para ello se utilizan las configuraciones `lock_timeout` y `statement_timeout`.

```
cursor.execute("SET LOCAL lock_timeout = '5s'")
```

```
cursor.execute("SET LOCAL statement_timeout = '15s'")
```

`lock_timeout` establece el tiempo máximo que una operación puede esperar para obtener un bloqueo sobre un registro. Si el bloqueo no se obtiene dentro del tiempo establecido, la operación se cancela.

`statement_timeout` define el tiempo máximo permitido para ejecutar una consulta. Si una consulta supera ese tiempo, PostgreSQL la cancela automáticamente.

Cuando una operación no puede completarse por espera de bloqueo o por exceder el tiempo permitido, el sistema revierte los cambios mediante `rollback()` y muestra un mensaje informativo al usuario.

```
except errors.LockNotAvailable:
```

```
    conexion.rollback()
```

```
    flash("La cuenta está siendo utilizada por otra operación. Intente nuevamente.", "warning")
```

except errors.QueryCanceled:

```
conexion.rollback()
```

```
flash("La operación tardó demasiado y fue cancelada. Intente nuevamente.",  
"warning")
```

Con estos mecanismos, el sistema evita esperas indefinidas, reduce el riesgo de inconsistencias y mantiene un control adecuado sobre las operaciones bancarias ejecutadas de forma simultánea.

Triggers de auditoría

Los triggers de auditoría permiten registrar automáticamente eventos importantes dentro del sistema. Estos se ejecutan directamente en PostgreSQL cuando ocurre una acción específica sobre una tabla, como insertar, actualizar o cambiar el estado de un registro.

En GBA CREDOMATIC, los triggers se utilizaron para registrar acciones como creación de usuarios, registro de clientes, creación de cuentas, activación o inactivación de usuarios, clientes y cuentas, procesamiento de transacciones, pagos de servicios y anulaciones.

Estos registros se almacenan en la tabla `auditoria`, permitiendo consultar posteriormente qué acción se realizó, en qué módulo ocurrió, qué usuario la ejecutó y en qué fecha sucedió.

Función de los triggers

Los triggers ayudan a mantener trazabilidad dentro del sistema. Esto significa que cada acción importante queda registrada automáticamente sin depender únicamente del código de la aplicación.

Por ejemplo, cuando un cajero registra un cliente, el trigger guarda un evento en auditoría indicando que se registró un nuevo cliente. De la misma forma, cuando se realiza una transferencia, el sistema registra el tipo de operación, el usuario responsable y una descripción del movimiento.

Ventajas de usar triggers

- Los triggers permiten automatizar el registro de eventos importantes.
- Ayudan a mantener evidencia de las acciones realizadas.
- Mejoran el control interno del sistema.
- Permiten al auditor revisar operaciones sensibles.
- Reducen el riesgo de olvidar registrar eventos desde el código de la aplicación.

Scripts de Triggers utilizados.

```
-- =====  
  
-- TRIGGERS DE AUDITORÍA CORREGIDOS - GBA CREDOMATIC  
  
-- Base de datos: Proyecto_Banco_Distribuido  
  
-- Tabla utilizada: public.auditoria  
  
-- =====  
  
  
  
-- =====  
  
-- 1. FUNCIONES AUXILIARES PARA OBTENER USUARIO DESDE FLASK  
  
-- =====  
  
  
  
CREATE OR REPLACE FUNCTION public.fn_obtener_id_usuario_app()  
  
RETURNS INTEGER  
  
LANGUAGE plpgsql
```

```

AS $$

DECLARE
    v_id_usuario TEXT;

BEGIN
    v_id_usuario := current_setting('app.id_usuario', true);

    IF v_id_usuario IS NULL OR v_id_usuario = '' THEN
        RETURN NULL;
    END IF;

    RETURN v_id_usuario::INTEGER;

EXCEPTION
    WHEN OTHERS THEN
        RETURN NULL;
END;
$$;

CREATE OR REPLACE FUNCTION public.fn_obtener_usuario_app()
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_usuario TEXT;

BEGIN

```

```

v_usuario := current_setting('app.usuario', true);

IF v_usuario IS NULL OR v_usuario = " THEN
    RETURN 'Sistema BD';
END IF;

RETURN v_usuario;

EXCEPTION
    WHEN OTHERS THEN
        RETURN 'Sistema BD';
END;
$$;

-- =====
-- 2. TRIGGER: REGISTRO DE USUARIOS INTERNOS
-- Registra cuando el administrador crea un usuario.
-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_insert_usuario()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO public.auditoria (

```

```

        id_usuario,
        modulo,
        operacion,
        descripcion
    )
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Usuarios',
    'REGISTRO_USUARIO',
    'El administrador ' || public.fn_obtener_usuario_app() ||
    ' registró al usuario interno ' || NEW.usuario || '
);

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_insert_usuario
AFTER INSERT ON public.usuarios
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_insert_usuario();

-- =====
-- 3. TRIGGER: ACTIVACIÓN / INACTIVACIÓN DE USUARIOS
-- Registra cuando un usuario cambia de ACTIVO a INACTIVO o viceversa.

```

-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_estado_usuario()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

BEGIN

IF OLD.estado IS DISTINCT FROM NEW.estado THEN

IF NEW.estado = 'INACTIVO' THEN

INSERT INTO public.auditoria (

id_usuario,

modulo,

operacion,

descripcion

)

VALUES (

public.fn_obtener_id_usuario_app(),

'Usuarios',

'INACTIVACION_USUARIO',

'El administrador ' || public.fn_obtener_usuario_app() ||

' inactivó al usuario interno ' || NEW.usuario || '

);

ELSIF NEW.estado = 'ACTIVO' THEN

INSERT INTO public.auditoria (


```

        id_usuario,
        modulo,
        operacion,
        descripcion
    )
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Usuarios',
    'ACTIVACION_USUARIO',
    'El administrador ' || public.fn_obtener_usuario_app() ||
    ' activó al usuario ' || NEW.usuario || ' '
);
END IF;

END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER trg_auditar_estado_usuario
AFTER UPDATE OF estado ON public.usuarios
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_estado_usuario();

```

```

-- =====
-- 4. TRIGGER: REGISTRO DE CLIENTES
-- Registra cuando el cajero crea un cliente.
-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_insert_cliente()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO public.auditoria (
        id_usuario,
        modulo,
        operacion,
        descripcion
    )
    VALUES (
        public.fn_obtener_id_usuario_app(),
        'Clientes',
        'INSERT',
        'El cajero ' || public.fn_obtener_usuario_app() ||
        ' registró al cliente ' || NEW.nombres || ' ' || NEW.apellidos ||
        ' con DPI ' || NEW.dpi || ' '
    );

    RETURN NEW;

```

```
END;
```

```
$$;
```

```
CREATE TRIGGER trg_auditar_insert_cliente
```

```
AFTER INSERT ON public.clientes
```

```
FOR EACH ROW
```

```
EXECUTE FUNCTION public.fn_auditar_insert_cliente();
```

```
-- =====
```

```
-- 5. TRIGGER: ACTIVACIÓN / INACTIVACIÓN DE CLIENTES
```

```
-- Registra cuando un cliente cambia de estado.
```

```
-- =====
```

```
CREATE OR REPLACE FUNCTION public.fn_auditar_estado_cliente()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF OLD.estado IS DISTINCT FROM NEW.estado THEN
```

```
        IF NEW.estado = 'INACTIVO' THEN
```

```
            INSERT INTO public.auditoria (
```

```
                id_usuario,
```

```
                modulo,
```

```
                operacion,
```

```

        descripcion
    )
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Clientes',
    'INACTIVACION_CLIENTE',
    'El administrador ' || public.fn_obtener_usuario_app() ||
    ' inactivó al cliente ' || NEW.nombres || ' ' || NEW.apellidos || ' '
);

ELSIF NEW.estado = 'ACTIVO' THEN
    INSERT INTO public.auditoria (
        id_usuario,
        modulo,
        operacion,
        descripcion
    )
VALUES (
        public.fn_obtener_id_usuario_app(),
        'Clientes',
        'ACTIVACION_CLIENTE',
        'El administrador ' || public.fn_obtener_usuario_app() ||
        ' activó al cliente ' || NEW.nombres || ' ' || NEW.apellidos || ' '
    );
END IF;

```

```

END IF;

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_estado_cliente
AFTER UPDATE OF estado ON public.clientes
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_estado_cliente();

-- =====
-- 6. TRIGGER: CREACIÓN DE CUENTAS
-- Registra cuando el cajero crea una cuenta bancaria.
-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_insert_cuenta()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO public.auditoria (
        id_usuario,
        modulo,
        operacion,

```

```

        descripcion
    )
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Cuentas',
    'INSERT',
    'El cajero ' || public.fn_obtener_usuario_app() ||
    ' creó la cuenta ' || NEW.numero_cuenta || ' '
);

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_insert_cuenta
AFTER INSERT ON public.cuentas
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_insert_cuenta();

```

```

-- =====
-- 7. TRIGGER: ACTIVACIÓN / INACTIVACIÓN DE CUENTAS
-- Registra cuando una cuenta cambia de estado.
-- =====

```

```

CREATE OR REPLACE FUNCTION public.fn_auditar_estado_cuenta()

```

```

RETURNS TRIGGER

LANGUAGE plpgsql

AS $$

BEGIN

    IF OLD.estado IS DISTINCT FROM NEW.estado THEN

        IF NEW.estado IN ('INACTIVA', 'INACTIVO') THEN

            INSERT INTO public.auditoria (

                id_usuario,

                modulo,

                operacion,

                descripcion

            )

            VALUES (

                public.fn_obtener_id_usuario_app(),

                'Cuentas',

                'INACTIVACION_CUENTA',

                'El administrador ' || public.fn_obtener_usuario_app() ||

                ' inactivó la cuenta ' || NEW.numero_cuenta || '

            );

        ELSEIF NEW.estado IN ('ACTIVA', 'ACTIVO') THEN

            INSERT INTO public.auditoria (

                id_usuario,

                modulo,

                operacion,

```

```

        descripcion
    )
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Cuentas',
    'ACTIVACION_CUENTA',
    'El administrador ' || public.fn_obtener_usuario_app() ||
    ' activó la cuenta ' || NEW.numero_cuenta || ' '
);
END IF;

END IF;

RETURN NEW;
END;
$$;

```

```

CREATE TRIGGER trg_auditar_estado_cuenta
AFTER UPDATE OF estado ON public.cuentas
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_estado_cuenta();

```

```

-- =====
-- 8. TRIGGER: REGISTRO DE TRANSACCIONES
-- Registra la operación real: DEPOSITO, RETIRO,

```


-- TRANSFERENCIA o PAGO_SERVICIO.

-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_insert_transaccion()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

DECLARE

 v_usuario TEXT;

 v_operacion TEXT;

 v_descripcion TEXT;

BEGIN

 v_usuario := public.fn_obtener_usuario_app();

 v_operacion := NEW.tipo_transaccion;

 IF NEW.tipo_transaccion = 'DEPOSITO' THEN

 v_descripcion :=

 'El cajero ' || v_usuario ||

 ' realizó un depósito por Q' || NEW.monto ||

 ' en la cuenta destino ID ' || COALESCE(NEW.id_cuenta_destino::TEXT, 'N/A') || ';

 ELSIF NEW.tipo_transaccion = 'RETIRO' THEN

 v_descripcion :=

 'El cajero ' || v_usuario ||

```

        ' realizó un retiro por Q' || NEW.monto ||
        ' desde la cuenta origen ID ' || COALESCE(NEW.id_cuenta_origen::TEXT, 'N/A') || ':';

ELSIF NEW.tipo_transaccion = 'TRANSFERENCIA' THEN

    v_descripcion :=
        'El cajero ' || v_usuario ||
        ' realizó una transferencia por Q' || NEW.monto ||
        ' desde la cuenta origen ID ' || COALESCE(NEW.id_cuenta_origen::TEXT, 'N/A') ||
        ' hacia la cuenta destino ID ' || COALESCE(NEW.id_cuenta_destino::TEXT, 'N/A') || ':';

ELSIF NEW.tipo_transaccion = 'PAGO_SERVICIO' THEN

    v_descripcion :=
        'El cajero ' || v_usuario ||
        ' registró una transacción de pago de servicio por Q' || NEW.monto ||
        ' desde la cuenta origen ID ' || COALESCE(NEW.id_cuenta_origen::TEXT, 'N/A') || ':';

ELSE

    v_operacion := 'OPERACION_BANCARIA';

    v_descripcion :=
        'El usuario ' || v_usuario ||
        ' procesó una operación bancaria por Q' || NEW.monto || ':';

```

```

END IF;

INSERT INTO public.auditoria (
    id_usuario,
    modulo,
    operacion,
    descripcion
)
VALUES (
    public.fn_obtener_id_usuario_app(),
    'Transacciones',
    v_operacion,
    v_descripcion
);

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_insert_transaccion
AFTER INSERT ON public.transacciones
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_insert_transaccion();

```

```
-- =====
```

```

-- 9. TRIGGER: ANULACIÓN DE TRANSACCIONES

-- Registra cuando una transacción cambia a estado ANULADA.

-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_estado_transaccion()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.estado IS DISTINCT FROM NEW.estado THEN

        IF NEW.estado = 'ANULADA' THEN

            INSERT INTO public.auditoria (
                id_usuario,
                modulo,
                operacion,
                descripcion
            )
            VALUES (
                public.fn_obtener_id_usuario_app(),
                'Transacciones',
                'ANULACION',
                'El cajero ' || public.fn_obtener_usuario_app() ||
                ' anuló la transacción ' || NEW.codigo_transaccion ||
                ' de tipo ' || NEW.tipo_transaccion ||

```

```

        ' por Q' || NEW.monto || '
    );

END IF;

END IF;

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_estado_transaccion
AFTER UPDATE OF estado ON public.transacciones
FOR EACH ROW
EXECUTE FUNCTION public.fn_auditar_estado_transaccion();

-- =====
-- 10. TRIGGER: REGISTRO DE PAGOS
-- Registra pagos con operación PAGO_SERVICIO.
-- =====

CREATE OR REPLACE FUNCTION public.fn_auditar_insert_pago()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$

```

```

BEGIN

INSERT INTO public.auditoria (

    id_usuario,

    modulo,

    operacion,

    descripcion

)

VALUES (

    public.fn_obtener_id_usuario_app(),

    'Pagos',

    'PAGO_SERVICIO',

    'El cajero ' || public.fn_obtener_usuario_app() ||

    ' registró un pago de servicio por Q' || NEW.monto ||

    ' con referencia ' || NEW.referencia || '

);

RETURN NEW;

END;

$$;

CREATE TRIGGER trg_auditar_insert_pago

AFTER INSERT ON public.pagos

FOR EACH ROW

EXECUTE FUNCTION public.fn_auditar_insert_pago();

```

```

-- =====
-- 11. CONSULTA DE VERIFICACIÓN
-- =====

SELECT
    id_auditoria,
    id_usuario,
    modulo,
    operacion,
    descripcion,
    fecha
FROM public.auditoria
ORDER BY id_auditoria DESC;

```

Respaldo y recuperación

El sistema GBA CREDOMATIC incorpora mecanismos de respaldo y recuperación con el objetivo de proteger la información almacenada en la base de datos. En un sistema bancario, la pérdida de datos puede afectar clientes, cuentas, transacciones, pagos y registros de auditoría, por lo que es necesario contar con estrategias que permitan recuperar información en caso de errores o fallos.

Para este proyecto se implementó un esquema de respaldo lógico dentro de PostgreSQL, utilizando un esquema llamado `backup_gba`. Este esquema permite almacenar copias de las tablas principales del sistema y conservar información respaldada para pruebas de recuperación.

Respaldo base inicial

El respaldo base inicial guarda una copia completa del estado actual de las tablas principales del sistema. Este respaldo incluye información de usuarios, clientes, cuentas, transacciones, pagos, roles, servicios, tipos de cuenta, auditoría y solicitudes de contacto.

Este respaldo sirve como punto de recuperación inicial. Si posteriormente ocurre una pérdida de datos en una tabla, es posible recuperar la información desde las tablas de respaldo.

Respaldo continuo lógico

Además del respaldo base, se implementó un respaldo continuo lógico mediante triggers. Este mecanismo registra los cambios realizados en tablas críticas, guardando información sobre inserciones, actualizaciones y eliminaciones.

El respaldo continuo permite conservar evidencia de los cambios realizados después del respaldo inicial. Esto ayuda a identificar qué registros fueron modificados y facilita el análisis de recuperación.

Recuperación de datos

La recuperación permite restaurar información eliminada o afectada utilizando los datos almacenados en las tablas de respaldo. Para comprobar el funcionamiento de esta estrategia, se realizaron pruebas eliminando datos de tablas específicas y restaurándolos desde el esquema backup_gba.

Durante estas pruebas se verificó que los datos podían recuperarse correctamente y que las secuencias SERIAL podían reajustarse para evitar errores en registros futuros.

Scripts de respaldo.

```
-- =====
```

```
-- RESPALDO BASE INICIAL - GBA CREDOMATIC
```



```

-- Base de datos: Proyecto_Banco_Distribuido

-- Tablas incluidas:

-- auditoria, clientes, cuentas, pagos, roles, servicios,

-- solicitudes_contacto, tipos_cuenta, transacciones, usuarios

-- =====

CREATE SCHEMA IF NOT EXISTS backup_gba;

-- =====

-- 1. TABLA MAESTRA DE RESPALDOS BASE

-- Guarda el historial de respaldos completos realizados.

-- =====

CREATE TABLE IF NOT EXISTS backup_gba.respaldos_base (

    id_respaldo_base SERIAL PRIMARY KEY,

    descripcion TEXT NOT NULL,

    fecha_respaldo TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

```

-- =====

-- 2. TABLAS DE RESPALDO BASE

-- Cada tabla copia la estructura de su tabla original,

-- agregando id_respaldo_base para identificar a qué respaldo pertenece.

-- =====

CREATE TABLE IF NOT EXISTS backup_gba.auditoria_base AS

SELECT NULL::INTEGER AS id_respaldo_base, a.*

FROM public.auditoria a

WITH NO DATA;

CREATE TABLE IF NOT EXISTS backup_gba.clientes_base AS

SELECT NULL::INTEGER AS id_respaldo_base, c.*

FROM public.clientes c

WITH NO DATA;

CREATE TABLE IF NOT EXISTS backup_gba.cuentas_base AS

```
SELECT NULL::INTEGER AS id_respaldo_base, c.*
```

```
FROM public.cuentas c
```

```
WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.pagos_base AS
```

```
SELECT NULL::INTEGER AS id_respaldo_base, p.*
```

```
FROM public.pagos p
```

```
WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.roles_base AS
```

```
SELECT NULL::INTEGER AS id_respaldo_base, r.*
```

```
FROM public.roles r
```

```
WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.servicios_base AS
```

```
SELECT NULL::INTEGER AS id_respaldo_base, s.*
```

```
FROM public.servicios s
```

```
WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.solicitudes_contacto_base AS

SELECT NULL::INTEGER AS id_respaldo_base, sc.*

FROM public.solicitudes_contacto sc

WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.tipos_cuenta_base AS

SELECT NULL::INTEGER AS id_respaldo_base, tc.*

FROM public.tipos_cuenta tc

WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.transacciones_base AS

SELECT NULL::INTEGER AS id_respaldo_base, t.*

FROM public.transacciones t

WITH NO DATA;
```

```
CREATE TABLE IF NOT EXISTS backup_gba.usuarios_base AS

SELECT NULL::INTEGER AS id_respaldo_base, u.*

FROM public.usuarios u

WITH NO DATA;
```

```

-- =====

-- 3. FUNCIÓN PARA CREAR RESPALDO BASE ACTUAL

-- Copia todos los datos actuales de las 10 tablas principales.

-- =====

CREATE OR REPLACE FUNCTION backup_gba.crear_respaldo_base_actual(

    p_descripcion TEXT DEFAULT 'Respaldo base inicial del sistema'

)

RETURNS INTEGER

LANGUAGE plpgsql

AS $$

DECLARE

    v_id_respaldo_base INTEGER;

BEGIN

    INSERT INTO backup_gba.respaldos_base (descripcion)

    VALUES (p_descripcion)

    RETURNING id_respaldo_base INTO v_id_respaldo_base;

```

```
INSERT INTO backup_gba.auditoria_base
```

```
SELECT v_id_respaldo_base, a.*
```

```
FROM public.auditoria a;
```

```
INSERT INTO backup_gba.clientes_base
```

```
SELECT v_id_respaldo_base, c.*
```

```
FROM public.clientes c;
```

```
INSERT INTO backup_gba.cuentas_base
```

```
SELECT v_id_respaldo_base, c.*
```

```
FROM public.cuentas c;
```

```
INSERT INTO backup_gba.pagos_base
```

```
SELECT v_id_respaldo_base, p.*
```

```
FROM public.pagos p;
```

```
INSERT INTO backup_gba.roles_base
```

```
SELECT v_id_respaldo_base, r.*
```

```
FROM public.roles r;
```

```
INSERT INTO backup_gba.servicios_base
```

```
SELECT v_id_respaldo_base, s.*
```

```
FROM public.servicios s;
```

```
INSERT INTO backup_gba.solicitudes_contacto_base
```

```
SELECT v_id_respaldo_base, sc.*
```

```
FROM public.solicitudes_contacto sc;
```

```
INSERT INTO backup_gba.tipos_cuenta_base
```

```
SELECT v_id_respaldo_base, tc.*
```

```
FROM public.tipos_cuenta tc;
```

```
INSERT INTO backup_gba.transacciones_base
```

```
SELECT v_id_respaldo_base, t.*
```

```
FROM public.transacciones t;
```

```
INSERT INTO backup_gba.usuarios_base
```

```

SELECT v_id_respaldo_base, u.*

FROM public.usuarios u;


RETURN v_id_respaldo_base;

END;

$$;


-- =====

-- 4. CREAR RESPALDO BASE INICIAL

-- Ejecuta el respaldo completo del estado actual.

-- =====


SELECT backup_gba.crear_respaldo_base_actual(

    'Respaldo base del sistema bancario'

) AS id_respaldo_base_creado;

```



```
SELECT *  
  
FROM backup_gba.respaldo_base  
  
ORDER BY id_respaldo_base DESC;
```

```
-- =====  
  
-- CONSULTAR DATOS RESPALDADOS DEL RESPALDO BASE #1  
  
-- =====
```

```
-- Auditoría respaldada
```

```
SELECT *  
  
FROM backup_gba.auditoria_base  
  
WHERE id_respaldo_base = 1  
  
ORDER BY id_auditoria ASC;
```

```
-- Clientes respaldados
```

```
SELECT *
```

```
FROM backup_gba.clientes_base
```

```
WHERE id_respaldo_base = 1
```

```
ORDER BY id_cliente ASC;
```

```
-- Cuentas respaldadas
```

```
SELECT *
```

```
FROM backup_gba.cuentas_base
```

```
WHERE id_respaldo_base = 1
```

```
ORDER BY id_cuenta ASC;
```

```
-- Pagos respaldados
```

```
SELECT *
```

```
FROM backup_gba.pagos_base
```

```
WHERE id_respaldo_base = 1
```

```
ORDER BY id_pago ASC;
```

-- Roles respaldados

SELECT *

FROM backup_gba.roles_base

WHERE id_respaldo_base = 1

ORDER BY id_rol ASC;

-- Servicios respaldados

SELECT *

FROM backup_gba.servicios_base

WHERE id_respaldo_base = 1

ORDER BY id_servicio ASC;

-- Solicitudes de contacto respaldadas

SELECT *

FROM backup_gba.solicitudes_contacto_base

WHERE id_respaldo_base = 1

ORDER BY id_solicitud ASC;

-- Tipos de cuenta respaldados

SELECT *

FROM backup_gba.tipos_cuenta_base

WHERE id_respaldo_base = 1

ORDER BY id_tipo_cuenta ASC;

-- Transacciones respaldadas

SELECT *

FROM backup_gba.transacciones_base

WHERE id_respaldo_base = 1

ORDER BY id_transaccion ASC;

-- Usuarios respaldados

SELECT *

FROM backup_gba.usuarios_base

```
WHERE id_respaldo_base = 1
```

```
ORDER BY id_usuario ASC;
```

```
-- =====
```

```
-- RESPALDO CONTINUO LÓGICO - GBA CREDOMATIC
```

```
-- Base de datos: Proyecto_Banco_Distribuido
```

```
-- Uso: PostgreSQL / pgAdmin / Query Tool
```

```
-- =====
```

```
-- =====
```

```
-- 1. CREAR ESQUEMA DE RESPALDO
```

```
-- Descripción:
```

```
-- Crea un espacio separado llamado backup_gba para guardar
```

```
-- información de respaldo sin mezclarla con las tablas principales.
```

```
-- =====
```

```
CREATE SCHEMA IF NOT EXISTS backup_gba;
```

```
-- =====

-- 2. TABLA DE CAMBIOS CONTINUOS

-- Descripción:

-- Guarda cada cambio realizado en tablas críticas.

-- Cada registro almacena:

-- - tabla afectada

-- - tipo de operación

-- - datos anteriores

-- - datos nuevos

-- - fecha del cambio

-- =====
```

```
CREATE TABLE IF NOT EXISTS backup_gba.cambios_continuos (

    id_cambio SERIAL PRIMARY KEY,

    tabla_afectada VARCHAR(80) NOT NULL,

    operacion VARCHAR(20) NOT NULL,

    id_registro INTEGER,

    datos_anteriores JSONB,

    datos_nuevos JSONB,
```

```

    fecha_cambio TIMESTAMP DEFAULT CURRENT_TIMESTAMP

);

-- =====

-- 3. FUNCIÓN GENERAL DE RESPALDO CONTINUO

-- Descripción:

-- Esta función se ejecuta automáticamente con triggers.

-- Guarda una copia lógica del registro insertado, actualizado o eliminado.

-- =====

CREATE OR REPLACE FUNCTION backup_gba.fn_respaldo_continuo()

RETURNS TRIGGER

LANGUAGE plpgsql

AS $$

DECLARE

    v_id_registro INTEGER;

BEGIN

    -- INSERT: cuando se agrega un nuevo registro

```

```
IF TG_OP = 'INSERT' THEN
```

```
    IF TG_TABLE_NAME = 'usuarios' THEN
```

```
        v_id_registro := NEW.id_usuario;
```

```
    ELSIF TG_TABLE_NAME = 'clientes' THEN
```

```
        v_id_registro := NEW.id_cliente;
```

```
    ELSIF TG_TABLE_NAME = 'cuentas' THEN
```

```
        v_id_registro := NEW.id_cuenta;
```

```
    ELSIF TG_TABLE_NAME = 'transacciones' THEN
```

```
        v_id_registro := NEW.id_transaccion;
```

```
    ELSIF TG_TABLE_NAME = 'pagos' THEN
```

```
        v_id_registro := NEW.id_pago;
```

```
    ELSIF TG_TABLE_NAME = 'auditoria' THEN
```

```
        v_id_registro := NEW.id_auditoria;
```

```
    ELSE
```

```
        v_id_registro := NULL;
```

```
    END IF;
```

```
    INSERT INTO backup_gba.cambios_continuos (
```



```

        tabla_afectada,

        operacion,

        id_registro,

        datos_anteriores,

        datos_nuevos

    )

VALUES (

    TG_TABLE_NAME,

    TG_OP,

    v_id_registro,

    NULL,

    to_jsonb(NEW)

);

RETURN NEW;

END IF;

```

-- UPDATE: cuando se modifica un registro

```
IF TG_OP = 'UPDATE' THEN

    IF TG_TABLE_NAME = 'usuarios' THEN

        v_id_registro := NEW.id_usuario;

    ELSIF TG_TABLE_NAME = 'clientes' THEN

        v_id_registro := NEW.id_cliente;

    ELSIF TG_TABLE_NAME = 'cuentas' THEN

        v_id_registro := NEW.id_cuenta;

    ELSIF TG_TABLE_NAME = 'transacciones' THEN

        v_id_registro := NEW.id_transaccion;

    ELSIF TG_TABLE_NAME = 'pagos' THEN

        v_id_registro := NEW.id_pago;

    ELSIF TG_TABLE_NAME = 'auditoria' THEN

        v_id_registro := NEW.id_auditoria;

    ELSE

        v_id_registro := NULL;

    END IF;

    INSERT INTO backup_gba.cambios_continuos (
```

```

        tabla_afectada,

        operacion,

        id_registro,

        datos_anteriores,

        datos_nuevos

    )

VALUES (

    TG_TABLE_NAME,

    TG_OP,

    v_id_registro,

    to_jsonb(OLD),

    to_jsonb(NEW)

);

RETURN NEW;

END IF;

-- DELETE: cuando se elimina un registro

```

```
IF TG_OP = 'DELETE' THEN
```

```
    IF TG_TABLE_NAME = 'usuarios' THEN
```

```
        v_id_registro := OLD.id_usuario;
```

```
    ELSIF TG_TABLE_NAME = 'clientes' THEN
```

```
        v_id_registro := OLD.id_cliente;
```

```
    ELSIF TG_TABLE_NAME = 'cuentas' THEN
```

```
        v_id_registro := OLD.id_cuenta;
```

```
    ELSIF TG_TABLE_NAME = 'transacciones' THEN
```

```
        v_id_registro := OLD.id_transaccion;
```

```
    ELSIF TG_TABLE_NAME = 'pagos' THEN
```

```
        v_id_registro := OLD.id_pago;
```

```
    ELSIF TG_TABLE_NAME = 'auditoria' THEN
```

```
        v_id_registro := OLD.id_auditoria;
```

```
    ELSE
```

```
        v_id_registro := NULL;
```

```
    END IF;
```

```
INSERT INTO backup_gba.cambios_continuos (
```

```
        tabla_afectada,  
  
        operacion,  
  
        id_registro,  
  
        datos_anteriores,  
  
        datos_nuevos  
    )  
  
    VALUES (  
  
        TG_TABLE_NAME,  
  
        TG_OP,  
  
        v_id_registro,  
  
        to_jsonb(OLD),  
  
        NULL  
    );  
  
  
    RETURN OLD;  
  
END IF;  
  
  
    RETURN NULL;  
  
END;
```

\$\$;

-- =====

-- 4. ELIMINAR TRIGGERS ANTERIORES SI EXISTEN

-- Descripción:

-- Es para evitar errores al ejecutar el script más de una vez.

-- =====

DROP TRIGGER IF EXISTS trg_respaldo_continuo_usuarios ON public.usuarios;

DROP TRIGGER IF EXISTS trg_respaldo_continuo_clientes ON public.clientes;

DROP TRIGGER IF EXISTS trg_respaldo_continuo_cuentas ON public.cuentas;

DROP TRIGGER IF EXISTS trg_respaldo_continuo_transacciones ON
public.transacciones;

DROP TRIGGER IF EXISTS trg_respaldo_continuo_pagos ON public.pagos;

DROP TRIGGER IF EXISTS trg_respaldo_continuo_auditoria ON public.auditoria;

-- =====

-- 5. CREAR TRIGGERS DE RESPALDO CONTINUO

-- Descripción:

-- Cada trigger se activa automáticamente cuando ocurre un

-- INSERT, UPDATE o DELETE en una tabla crítica.

-- =====

```
CREATE TRIGGER trg_respaldo_continuo_usuarios
AFTER INSERT OR UPDATE OR DELETE ON public.usuarios
FOR EACH ROW
EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();
```

```
CREATE TRIGGER trg_respaldo_continuo_clientes
AFTER INSERT OR UPDATE OR DELETE ON public.clientes
FOR EACH ROW
EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();
```

```
CREATE TRIGGER trg_respaldo_continuo_cuentas
```

AFTER INSERT OR UPDATE OR DELETE ON public.cuentas

FOR EACH ROW

EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();

CREATE TRIGGER trg_respaldo_continuo_transacciones

AFTER INSERT OR UPDATE OR DELETE ON public.transacciones

FOR EACH ROW

EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();

CREATE TRIGGER trg_respaldo_continuo_pagos

AFTER INSERT OR UPDATE OR DELETE ON public.pagos

FOR EACH ROW

EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();

CREATE TRIGGER trg_respaldo_continuo_auditoria

AFTER INSERT OR UPDATE OR DELETE ON public.auditoria


```
FOR EACH ROW
```

```
EXECUTE FUNCTION backup_gba.fn_respaldo_continuo();
```

```
-- =====
```

```
-- 6. CONSULTA DE VERIFICACIÓN
```

```
-- Descripción:
```

```
-- Permite ver los últimos cambios respaldados.
```

```
-- =====
```

```
SELECT *
```

```
FROM backup_gba.cambios_continuos
```

```
ORDER BY id_cambio DESC;
```

```
-- =====
```

```
-- Para ver los cambios respaldados
```

```
-- =====
```

```
SELECT
```

```
    id_cambio,
```

```
    tabla_afectada,
```

```
    operacion,
```

```
    id_registro,
```

```
    fecha_cambio
```

```
FROM backup_gba.cambios_continuos
```

```
ORDER BY id_cambio DESC;
```

```
-- =====
```

```
-- Para ver el detalle completo de un cambio
```

```
-- =====
```

```
SELECT
```

```
    tabla_afectada,
```

```
    operacion,
```

```
    datos_anteriores,
```

```
    datos_nuevos,
```

```
    fecha_cambio
```

```
FROM backup_gba.cambios_continuos
```

```
ORDER BY id_cambio DESC;
```

```
-- =====
```

```
-- CONFIGURACIÓN WAL / PITR - POSTGRESQL
```

```
-- Descripción:
```

```
-- Activa configuración necesaria para archivado de WAL,
```

```
-- base para recuperación a punto en el tiempo.
```

```
-- =====
```

```
ALTER SYSTEM SET wal_level = 'replica';
```

```
ALTER SYSTEM SET archive_mode = 'on';
```

```
ALTER SYSTEM SET archive_timeout = '60s';
```

```
-- =====
```

```
-- CONFIGURACION POSTGRE DE COMO DEBE GUARDAR LOS WAL ARCHIVADOS
```

```
-- =====
```

```
ALTER SYSTEM SET archive_command = 'test ! -f /var/lib/postgresql/wal_archive/%f &&  
cp %p /var/lib/postgresql/wal_archive/%f';
```

```
-- =====
```

```
-- ESTO SOLAMENTE RECARGA LA CONFIGURACION SIN CERRAR PGADMIN
```

```
-- =====
```

```
SELECT pg_reload_conf();
```

Scripts de recuperación.

```
-- =====
```

```
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE AUDITORÍA
```

```
-- Respaldo usado: id_respaldo_base = 1
```

```
-- =====
```

```
-- 1. Ver datos actuales
```

```
SELECT COUNT(*) AS total_antes_eliminar
```

```
FROM public.auditoria;
```

```
SELECT *  
  
FROM public.auditoria  
  
ORDER BY id_auditoria ASC;
```

-- 2. Eliminar datos de la tabla auditoria

```
DELETE FROM public.auditoria;
```

-- 3. Confirmar que se eliminaron

```
SELECT COUNT(*) AS total_despues_eliminar  
  
FROM public.auditoria;
```

-- 4. Restaurar datos desde el respaldo base

```
INSERT INTO public.auditoria (  
  
    id_auditoria,  
  
    id_usuario,
```

```

        modulo,

        operacion,

        descripcion,

        fecha
    )

SELECT

    id_auditoria,

    id_usuario,

    modulo,

    operacion,

    descripcion,

    fecha

FROM backup_gba.auditoria_base

WHERE id_respaldo_base = 1;

```

-- 5. Reajustar secuencia SERIAL

```

SELECT setval(

    'auditoria_id_auditoria_seq',

```

```
COALESCE((SELECT MAX(id_auditoria) FROM public.auditoria), 1),  
  
true  
  
);
```

```
-- 6. Verificar recuperación
```

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.auditoria;
```

```
SELECT *  
  
FROM public.auditoria  
  
ORDER BY id_auditoria ASC;
```

```
-- =====
```

```
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE PAGOS
```

-- Respaldo usado: id_respaldo_base = 1

-- =====

-- 1. Ver datos actuales

SELECT COUNT(*) AS total_antes_eliminar

FROM public.pagos;

SELECT *

FROM public.pagos

ORDER BY id_pago ASC;

-- 2. Eliminar datos de la tabla pagos

DELETE FROM public.pagos;

-- 3. Confirmar que se eliminaron

SELECT COUNT(*) AS total_despues_eliminar

FROM public.pagos;

-- 4. Restaurar datos desde el respaldo base

```
INSERT INTO public.pagos (
```

```
    id_pago,
```

```
    codigo_pago,
```

```
    id_cuenta,
```

```
    id_servicio,
```

```
    referencia,
```

```
    monto,
```

```
    estado,
```

```
    fecha_pago
```

```
)
```

```
SELECT
```

```
    id_pago,
```

```
    codigo_pago,
```

```
    id_cuenta,
```

```
    id_servicio,
```

```
    referencia,
```

```
    monto,  
  
    estado,  
  
    fecha_pago  
  
FROM backup_gba.pagos_base  
  
WHERE id_respaldo_base = 1;
```

-- 5. Reajustar secuencia SERIAL

```
SELECT setval(  
  
    'pagos_id_pago_seq',  
  
    COALESCE((SELECT MAX(id_pago) FROM public.pagos), 1),  
  
    true  
  
);
```

-- 6. Verificar recuperación

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.pagos;
```

```
-- =====  
  
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE TRANSACCIONES  
  
-- Respaldo usado: id_respaldo_base = 1  
  
-- =====  
  
-- 1. Ver datos actuales  
  
SELECT COUNT(*) AS total_antes_eliminar  
  
FROM public.transacciones;  
  
  
SELECT *  
  
FROM public.transacciones  
  
ORDER BY id_transaccion ASC;
```

-- 2. Eliminar datos de la tabla transacciones

```
DELETE FROM public.transacciones;
```

-- 3. Confirmar que se eliminaron

```
SELECT COUNT(*) AS total_despues_eliminar
```

```
FROM public.transacciones;
```

-- 4. Restaurar datos desde el respaldo base

```
INSERT INTO public.transacciones (
```

```
    id_transaccion,
```

```
    codigo_transaccion,
```

```
    tipo_transaccion,
```

```
    id_cuenta_origen,
```

```
    id_cuenta_destino,
```

```
    monto,
```

```
    estado,
```

```
    descripcion,
```

```

        fecha_transaccion
    )

SELECT

    id_transaccion,

    codigo_transaccion,

    tipo_transaccion,

    id_cuenta_origen,

    id_cuenta_destino,

    monto,

    estado,

    descripcion,

    fecha_transaccion

FROM backup_gba.transacciones_base

WHERE id_respaldo_base = 1;

```

-- 5. Reajustar secuencia SERIAL

```

SELECT setval(

    'transacciones_id_transaccion_seq',

```

```
COALESCE((SELECT MAX(id_transaccion) FROM public.transacciones), 1),  
  
true  
  
);
```

```
-- 6. Verificar recuperación
```

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.transacciones;
```

```
-- =====
```

```
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE SERVICIOS
```

```
-- Respaldo usado: id_respaldo_base = 1
```

```
-- =====
```

```
-- 1. Ver datos actuales
```

```
SELECT COUNT(*) AS total_antes_eliminar
```

```
FROM public.servicios;
```

```
SELECT *
```

```
FROM public.servicios
```

```
ORDER BY id_servicio ASC;
```

```
-- 2. Eliminar datos dependientes de la tabla servicios
```

```
DELETE FROM public.pagos;
```

```
-- 3. Eliminar datos de la tabla servicios
```

```
DELETE FROM public.servicios;
```

```
-- 4. Confirmar que se eliminaron
```

```
SELECT COUNT(*) AS total_despues_eliminar
```

```
FROM public.servicios;
```

-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.servicios (
```

```
    id_servicio,
```

```
    nombre_servicio,
```

```
    descripcion,
```

```
    estado
```

```
)
```

```
SELECT
```

```
    id_servicio,
```

```
    nombre_servicio,
```

```
    descripcion,
```

```
    estado
```

```
FROM backup_gba.servicios_base
```

```
WHERE id_respaldo_base = 1;
```

-- 6. Restaurar datos dependientes desde el respaldo base


```
INSERT INTO public.pagos (
```

```
    id_pago,
```

```
    codigo_pago,
```

```
    id_cuenta,
```

```
    id_servicio,
```

```
    referencia,
```

```
    monto,
```

```
    estado,
```

```
    fecha_pago
```

```
)
```

```
SELECT
```

```
    id_pago,
```

```
    codigo_pago,
```

```
    id_cuenta,
```

```
    id_servicio,
```

```
    referencia,
```

```
    monto,
```

```
    estado,
```

```
    fecha_pago
```

```
FROM backup_gba.pagos_base
```

```
WHERE id_respaldo_base = 1;
```

```
-- 7. Reajustar secuencias SERIAL
```

```
SELECT setval(
```

```
    'servicios_id_servicio_seq',
```

```
    COALESCE((SELECT MAX(id_servicio) FROM public.servicios), 1),
```

```
    true
```

```
);
```

```
SELECT setval(
```

```
    'pagos_id_pago_seq',
```

```
    COALESCE((SELECT MAX(id_pago) FROM public.pagos), 1),
```

```
    true
```

```
);
```

```
-- 8. Verificar recuperación
```

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.servicios;
```

```
-- =====
```

```
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE CUENTAS
```

```
-- Respaldo usado: id_respaldo_base = 1
```

```
-- =====
```

```
-- 1. Ver datos actuales
```

```
SELECT COUNT(*) AS total_antes_eliminar  
  
FROM public.cuentas;
```

```
SELECT *  
  
FROM public.cuentas  
  
ORDER BY id_cuenta ASC;
```

-- 2. Eliminar datos dependientes de la tabla cuentas

```
DELETE FROM public.pagos;  
  
DELETE FROM public.transacciones;
```

-- 3. Eliminar datos de la tabla cuentas

```
DELETE FROM public.cuentas;
```

-- 4. Confirmar que se eliminaron

```
SELECT COUNT(*) AS total_despues_eliminar  
  
FROM public.cuentas;
```

-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.cuentas (
```

```
    id_cuenta,
```

```
    id_cliente,
```

```
    id_tipo_cuenta,
```

```
    numero_cuenta,
```

```
    saldo,
```

```
    estado,
```

```
    fecha_apertura
```

```
)
```

```
SELECT
```

```
    id_cuenta,
```

```
    id_cliente,
```

```
    id_tipo_cuenta,
```

```
    numero_cuenta,
```

```
    saldo,
```

```
    estado,
```

```
    fecha_apertura
```

```
FROM backup_gba.cuentas_base
```

```
WHERE id_respaldo_base = 1;
```

```
-- 6. Restaurar datos dependientes desde el respaldo base
```

```
INSERT INTO public.transacciones (
```

```
    id_transaccion,
```

```
    codigo_transaccion,
```

```
    tipo_transaccion,
```

```
    id_cuenta_origen,
```

```
    id_cuenta_destino,
```

```
    monto,
```

```
    estado,
```

```
    descripcion,
```

```
    fecha_transaccion
```

```
)
```

```
SELECT
```

```
    id_transaccion,
```

```
    codigo_transaccion,
```

```
    tipo_transaccion,
```

id_cuenta_origen,

id_cuenta_destino,

monto,

estado,

descripcion,

fecha_transaccion

FROM backup_gba.transacciones_base

WHERE id_respaldo_base = 1;

INSERT INTO public.pagos (

id_pago,

codigo_pago,

id_cuenta,

id_servicio,

referencia,

monto,

estado,

fecha_pago

)

```
SELECT

    id_pago,

    codigo_pago,

    id_cuenta,

    id_servicio,

    referencia,

    monto,

    estado,

    fecha_pago

FROM backup_gba.pagos_base

WHERE id_respaldo_base = 1;
```

-- 7. Reajustar secuencias SERIAL

```
SELECT setval(

    'cuentas_id_cuenta_seq',

    COALESCE((SELECT MAX(id_cuenta) FROM public.cuentas), 1),

    true

);
```



```
SELECT setval(  
  
    'transacciones_id_transaccion_seq',  
  
    COALESCE((SELECT MAX(id_transaccion) FROM public.transacciones), 1),  
  
    true  
  
);
```

```
SELECT setval(  
  
    'pagos_id_pago_seq',  
  
    COALESCE((SELECT MAX(id_pago) FROM public.pagos), 1),  
  
    true  
  
);
```

-- 8. Verificar recuperación

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.cuentas;
```

```
-- =====  
  
-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE CLIENTES  
  
-- Respaldo usado: id_respaldo_base = 1  
  
-- =====  
  
-- 1. Ver datos actuales  
  
SELECT COUNT(*) AS total_antes_eliminar  
  
FROM public.clientes;  
  
  
SELECT *  
  
FROM public.clientes  
  
ORDER BY id_cliente ASC;
```

-- 2. Eliminar datos dependientes de la tabla clientes

```
DELETE FROM public.pagos;
```

```
DELETE FROM public.transacciones;
```

```
DELETE FROM public.cuentas;
```

-- 3. Eliminar datos de la tabla clientes

```
DELETE FROM public.clientes;
```

-- 4. Confirmar que se eliminaron

```
SELECT COUNT(*) AS total_despues_eliminar
```

```
FROM public.clientes;
```

-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.clientes (
```

```
    id_cliente,
```

```
    dpi,
```

```
    nombres,  
  
    apellidos,  
  
    telefono,  
  
    correo,  
  
    direccion,  
  
    estado,  
  
    fecha_registro  
)  
  
SELECT  
  
    id_cliente,  
  
    dpi,  
  
    nombres,  
  
    apellidos,  
  
    telefono,  
  
    correo,  
  
    direccion,  
  
    estado,  
  
    fecha_registro  
  
FROM backup_gba.clientes_base
```

```
WHERE id_respaldo_base = 1;
```

```
-- 6. Restaurar datos dependientes desde el respaldo base
```

```
INSERT INTO public.cuentas (
```

```
    id_cuenta,
```

```
    id_cliente,
```

```
    id_tipo_cuenta,
```

```
    numero_cuenta,
```

```
    saldo,
```

```
    estado,
```

```
    fecha_apertura
```

```
)
```

```
SELECT
```

```
    id_cuenta,
```

```
    id_cliente,
```

```
    id_tipo_cuenta,
```

```
    numero_cuenta,
```

```
    saldo,
```

estado,

fecha_apertura

FROM backup_gba.cuentas_base

WHERE id_respaldo_base = 1;

INSERT INTO public.transacciones (

id_transaccion,

codigo_transaccion,

tipo_transaccion,

id_cuenta_origen,

id_cuenta_destino,

monto,

estado,

descripcion,

fecha_transaccion

)

SELECT

id_transaccion,

codigo_transaccion,

tipo_transaccion,

id_cuenta_origen,

id_cuenta_destino,

monto,

estado,

descripcion,

fecha_transaccion

FROM backup_gba.transacciones_base

WHERE id_respaldo_base = 1;

INSERT INTO public.pagos (

id_pago,

codigo_pago,

id_cuenta,

id_servicio,

referencia,

monto,

estado,

fecha_pago

)

SELECT

id_pago,

codigo_pago,

id_cuenta,

id_servicio,

referencia,

monto,

estado,

fecha_pago

FROM backup_gba.pagos_base

WHERE id_respaldo_base = 1;

-- 7. Reajustar secuencias SERIAL

SELECT setval(

'clientes_id_cliente_seq',

COALESCE((SELECT MAX(id_cliente) FROM public.clientes), 1),

true


```
);
```

```
SELECT setval(
```

```
    'cuentas_id_cuenta_seq',
```

```
    COALESCE((SELECT MAX(id_cuenta) FROM public.cuentas), 1),
```

```
    true
```

```
);
```

```
SELECT setval(
```

```
    'transacciones_id_transaccion_seq',
```

```
    COALESCE((SELECT MAX(id_transaccion) FROM public.transacciones), 1),
```

```
    true
```

```
);
```

```
SELECT setval(
```

```
    'pagos_id_pago_seq',
```

```
    COALESCE((SELECT MAX(id_pago) FROM public.pagos), 1),
```

```
    true
```

```
);
```

-- 8. Verificar recuperación

SELECT COUNT(*) AS total_despues_recuperar

FROM public.clientes;

-- =====

-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE USUARIOS

-- Respaldo usado: id_respaldo_base = 1

-- =====

-- 1. Ver datos actuales

SELECT COUNT(*) AS total_antes_eliminar

FROM public.usuarios;

```
SELECT *  
  
FROM public.usuarios  
  
ORDER BY id_usuario ASC;
```

-- 2. Eliminar datos dependientes de la tabla usuarios

```
DELETE FROM public.auditoria;
```

-- 3. Eliminar datos de la tabla usuarios

```
DELETE FROM public.usuarios;
```

-- 4. Confirmar que se eliminaron

```
SELECT COUNT(*) AS total_despues_eliminar  
  
FROM public.usuarios;
```

-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.usuarios (
```

```
    id_usuario,
```

```
    id_rol,
```

```
    nombre_completo,
```

```
    usuario,
```

```
    correo,
```

```
    clave,
```

```
    estado,
```

```
    fecha_registro
```

```
)
```

```
SELECT
```

```
    id_usuario,
```

```
    id_rol,
```

```
    nombre_completo,
```

```
    usuario,
```

```
    correo,
```

```
    clave,
```

```
    estado,
```

fecha_registro

FROM backup_gba.usuarios_base

WHERE id_respaldo_base = 1;

-- 6. Restaurar datos dependientes desde el respaldo base

INSERT INTO public.auditoria (

id_auditoria,

id_usuario,

modulo,

operacion,

descripcion,

fecha

)

SELECT

id_auditoria,

id_usuario,

modulo,

operacion,

descripcion,

fecha

FROM backup_gba.auditoria_base

WHERE id_respaldo_base = 1;

-- 7. Reajustar secuencias SERIAL

SELECT setval(

'usuarios_id_usuario_seq',

COALESCE((SELECT MAX(id_usuario) FROM public.usuarios), 1),

true

);

SELECT setval(

'auditoria_id_auditoria_seq',

COALESCE((SELECT MAX(id_auditoria) FROM public.auditoria), 1),

true

);

-- 8. Verificar recuperación

```
SELECT COUNT(*) AS total_despues_recuperar  
  
FROM public.usuarios;
```

-- =====

-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE ROLES

-- Respaldo usado: id_respaldo_base = 1

-- =====

-- 1. Ver datos actuales

```
SELECT COUNT(*) AS total_antes_eliminar  
  
FROM public.roles;
```

```
SELECT *
```

```
FROM public.roles
```

```
ORDER BY id_rol ASC;
```

```
-- 2. Eliminar datos dependientes de la tabla roles
```

```
DELETE FROM public.auditoria;
```

```
DELETE FROM public.usuarios;
```

```
-- 3. Eliminar datos de la tabla roles
```

```
DELETE FROM public.roles;
```

```
-- 4. Confirmar que se eliminaron
```

```
SELECT COUNT(*) AS total_despues_eliminar
```

```
FROM public.roles;
```


-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.roles (
```

```
    id_rol,
```

```
    nombre_rol,
```

```
    descripcion,
```

```
    estado,
```

```
    fecha_registro
```

```
)
```

```
SELECT
```

```
    id_rol,
```

```
    nombre_rol,
```

```
    descripcion,
```

```
    estado,
```

```
    fecha_registro
```

```
FROM backup_gba.roles_base
```

```
WHERE id_respaldo_base = 1;
```

-- 6. Restaurar datos dependientes desde el respaldo base

```
INSERT INTO public.usuarios (
```

```
    id_usuario,
```

```
    id_rol,
```

```
    nombre_completo,
```

```
    usuario,
```

```
    correo,
```

```
    clave,
```

```
    estado,
```

```
    fecha_registro
```

```
)
```

```
SELECT
```

```
    id_usuario,
```

```
    id_rol,
```

```
    nombre_completo,
```

```
    usuario,
```

```
    correo,
```

```
    clave,
```

```
    estado,
```

```
    fecha_registro
```

```
FROM backup_gba.usuarios_base
```

```
WHERE id_respaldo_base = 1;
```

```
INSERT INTO public.auditoria (
```

```
    id_auditoria,
```

```
    id_usuario,
```

```
    modulo,
```

```
    operacion,
```

```
    descripcion,
```

```
    fecha
```

```
)
```

```
SELECT
```

```
    id_auditoria,
```

```
    id_usuario,
```

```
    modulo,
```

```
    operacion,
```

```
    descripcion,
```

```
    fecha
```

```
FROM backup_gba.auditoria_base
```

```
WHERE id_respaldo_base = 1;
```

```
-- 7. Reajustar secuencias SERIAL
```

```
SELECT setval(  
  
    'roles_id_rol_seq',  
  
    COALESCE((SELECT MAX(id_rol) FROM public.roles), 1),  
  
    true  
  
);
```

```
SELECT setval(  
  
    'usuarios_id_usuario_seq',  
  
    COALESCE((SELECT MAX(id_usuario) FROM public.usuarios), 1),  
  
    true  
  
);
```

```
SELECT setval(  
  
    'auditoria_id_auditoria_seq',  
  
    COALESCE((SELECT MAX(id_auditoria) FROM public.auditoria), 1),
```

true

);

-- 8. Verificar recuperación

SELECT COUNT(*) AS total_despues_recuperar

FROM public.roles;

-- =====

-- PRUEBA DE ELIMINACIÓN Y RECUPERACIÓN DE TIPOS_CUENTA

-- Respaldo usado: id_respaldo_base = 1

-- =====

-- 1. Ver datos actuales

SELECT COUNT(*) AS total_antes_eliminar

FROM public.tipos_cuenta;

```
SELECT *
```

```
FROM public.tipos_cuenta
```

```
ORDER BY id_tipo_cuenta ASC;
```

```
-- 2. Eliminar datos dependientes de la tabla tipos_cuenta
```

```
DELETE FROM public.pagos;
```

```
DELETE FROM public.transacciones;
```

```
DELETE FROM public.cuentas;
```

```
-- 3. Eliminar datos de la tabla tipos_cuenta
```

```
DELETE FROM public.tipos_cuenta;
```

```
-- 4. Confirmar que se eliminaron
```

```
SELECT COUNT(*) AS total_despues_eliminar
```

```
FROM public.tipos_cuenta;
```

-- 5. Restaurar datos desde el respaldo base

```
INSERT INTO public.tipos_cuenta (
```

```
    id_tipo_cuenta,
```

```
    nombre_tipo,
```

```
    descripcion,
```

```
    estado
```

```
)
```

```
SELECT
```

```
    id_tipo_cuenta,
```

```
    nombre_tipo,
```

```
    descripcion,
```

```
    estado
```

```
FROM backup_gba.tipos_cuenta_base
```

```
WHERE id_respaldo_base = 1;
```

-- 6. Restaurar datos dependientes desde el respaldo base

```
INSERT INTO public.cuentas (  
  
    id_cuenta,  
  
    id_cliente,  
  
    id_tipo_cuenta,  
  
    numero_cuenta,  
  
    saldo,  
  
    estado,  
  
    fecha_apertura  
  
)  
  
SELECT  
  
    id_cuenta,  
  
    id_cliente,  
  
    id_tipo_cuenta,  
  
    numero_cuenta,  
  
    saldo,  
  
    estado,  
  
    fecha_apertura  
  
FROM backup_gba.cuentas_base  
  
WHERE id_respaldo_base = 1;
```



```
INSERT INTO public.transacciones (
```

```
    id_transaccion,
```

```
    codigo_transaccion,
```

```
    tipo_transaccion,
```

```
    id_cuenta_origen,
```

```
    id_cuenta_destino,
```

```
    monto,
```

```
    estado,
```

```
    descripcion,
```

```
    fecha_transaccion
```

```
)
```

```
SELECT
```

```
    id_transaccion,
```

```
    codigo_transaccion,
```

```
    tipo_transaccion,
```

```
    id_cuenta_origen,
```

```
    id_cuenta_destino,
```

```
    monto,
```

estado,

descripcion,

fecha_transaccion

FROM backup_gba.transacciones_base

WHERE id_respaldo_base = 1;

INSERT INTO public.pagos (

id_pago,

codigo_pago,

id_cuenta,

id_servicio,

referencia,

monto,

estado,

fecha_pago

)

SELECT

id_pago,

codigo_pago,

id_cuenta,

id_servicio,

referencia,

monto,

estado,

fecha_pago

FROM backup_gba.pagos_base

WHERE id_respaldo_base = 1;

-- 7. Reajustar secuencias SERIAL

SELECT setval(

'tipos_cuenta_id_tipo_cuenta_seq',

COALESCE((SELECT MAX(id_tipo_cuenta) FROM public.tipos_cuenta), 1),

true

);

SELECT setval(

'cuentas_id_cuenta_seq',

```

        COALESCE((SELECT MAX(id_cuenta) FROM public.cuentas), 1),

        true

    );

SELECT setval(

    'transacciones_id_transaccion_seq',

    COALESCE((SELECT MAX(id_transaccion) FROM public.transacciones), 1),

    true

);

SELECT setval(

    'pagos_id_pago_seq',

    COALESCE((SELECT MAX(id_pago) FROM public.pagos), 1),

    true

);

```

-- 8. Verificar recuperación

```
SELECT COUNT(*) AS total_despues_recuperar
```

```
FROM public.tipos_cuenta;
```

Optimización de consultas

La optimización de consultas permite mejorar el rendimiento de la base de datos al consultar información frecuente o crítica. En el sistema GBA CREDOMATIC, se trabajaron consultas relacionadas con historial de movimientos, estados de cuenta, auditoría, reportes de transacciones y clientes con mayor actividad financiera.

Para mejorar estas consultas se crearon índices sobre columnas utilizadas en filtros, relaciones y ordenamientos. Los índices permiten que PostgreSQL localice información de forma más rápida, especialmente cuando las tablas contienen una gran cantidad de registros.

Uso de índices

Los índices creados ayudan a mejorar búsquedas por número de cuenta, DPI, cuenta origen, cuenta destino, módulo, operación, fecha y tipo de transacción.

Aunque en una base de datos pequeña PostgreSQL puede seguir utilizando lectura secuencial, los índices preparan el sistema para mantener buen rendimiento cuando aumente el volumen de datos.

Scripts de consultas

```
-- =====  
  
-- CONSULTA 1: HISTORIAL DE MOVIMIENTOS POR CUENTA  
  
-- Sirve para ver depósitos, retiros, transferencias y pagos  
  
-- donde participa una cuenta.  
  
-- =====
```

-- Consulta normal

SELECT

t.codigo_transaccion,

t.tipo_transaccion,

co.numero_cuenta AS cuenta_origen,

cd.numero_cuenta AS cuenta_destino,

t.monto,

t.estado,

t.fecha_transaccion

FROM public.transacciones t

LEFT JOIN public.cuentas co ON t.id_cuenta_origen = co.id_cuenta

LEFT JOIN public.cuentas cd ON t.id_cuenta_destino = cd.id_cuenta

WHERE co.numero_cuenta = '1000010001'

OR cd.numero_cuenta = '1000010001'

ORDER BY t.fecha_transaccion DESC;

-- Índices para optimizar

CREATE INDEX IF NOT EXISTS idx_cuentas_numero_cuenta

ON public.cuentas (numero_cuenta);

CREATE INDEX IF NOT EXISTS idx_transacciones_origen_fecha

ON public.transacciones (id_cuenta_origen, fecha_transaccion DESC);

CREATE INDEX IF NOT EXISTS idx_transacciones_destino_fecha

```
ON public.transacciones (id_cuenta_destino, fecha_transaccion DESC);
```

```
-- Análisis de rendimiento
```

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    t.codigo_transaccion,
```

```
    t.tipo_transaccion,
```

```
    co.numero_cuenta AS cuenta_origen,
```

```
    cd.numero_cuenta AS cuenta_destino,
```

```
    t.monto,
```

```
    t.estado,
```

```
    t.fecha_transaccion
```

```
FROM public.transacciones t
```

```
LEFT JOIN public.cuentas co ON t.id_cuenta_origen = co.id_cuenta
```

```
LEFT JOIN public.cuentas cd ON t.id_cuenta_destino = cd.id_cuenta
```

```
WHERE co.numero_cuenta = '1000010001'
```

```
    OR cd.numero_cuenta = '1000010001'
```

```
ORDER BY t.fecha_transaccion DESC;
```

```
-- =====
```

```
-- CONSULTA 2: ESTADO DE CUENTA DE UN CLIENTE
```

```
-- Sirve para ver las cuentas, tipo de cuenta y saldo de un cliente.
```

```
-- =====
```

-- Consulta normal

SELECT

cl.dpi,

cl.nombres,

cl.apellidos,

c.numero_cuenta,

tc.nombre_tipo AS tipo_cuenta,

c.saldo,

c.estado,

c.fecha_apertura

FROM public.clientes cl

INNER JOIN public.cuentas c ON cl.id_cliente = c.id_cliente

INNER JOIN public.tipos_cuenta tc ON c.id_tipo_cuenta = tc.id_tipo_cuenta

WHERE cl.dpi = '78654326789057'

ORDER BY c.fecha_apertura DESC;

-- Índices para optimizar

CREATE INDEX IF NOT EXISTS idx_clientes_dpi

ON public.clientes (dpi);

CREATE INDEX IF NOT EXISTS idx_cuentas_cliente_fecha

ON public.cuentas (id_cliente, fecha_apertura DESC);

-- Análisis de rendimiento

EXPLAIN ANALYZE

SELECT

cl.dpi,

cl.nombres,

cl.apellidos,

c.numero_cuenta,

tc.nombre_tipo AS tipo_cuenta,

c.saldo,

c.estado,

c.fecha_apertura

FROM public.clientes cl

INNER JOIN public.cuentas c ON cl.id_cliente = c.id_cliente

INNER JOIN public.tipos_cuenta tc ON c.id_tipo_cuenta = tc.id_tipo_cuenta

WHERE cl.dpi = '09856142356718'

ORDER BY c.fecha_apertura DESC;

-- =====

-- CONSULTA 3: AUDITORÍA POR OPERACIÓN

-- Sirve para revisar eventos específicos del sistema.

-- Como los retiros registrados en auditoría.

-- =====

-- Consulta normal

```
SELECT
    a.fecha,
    COALESCE(u.usuario, 'cajero_banco') AS usuario,
    a.modulo,
    a.operacion,
    a.descripcion
FROM public.auditoria a
LEFT JOIN public.usuarios u ON a.id_usuario = u.id_usuario
WHERE a.modulo = 'Transacciones'
    AND a.operacion = 'RETIRO'
ORDER BY a.fecha DESC;
```

-- Índice para optimizar

```
CREATE INDEX IF NOT EXISTS idx_auditoria_modulo_operacion_fecha
ON public.auditoria (modulo, operacion, fecha DESC);
```

-- Análisis de rendimiento

```
EXPLAIN ANALYZE
```

```
SELECT
    a.fecha,
    COALESCE(u.usuario, 'cajero_banco') AS usuario,
    a.modulo,
    a.operacion,
    a.descripcion
```

```

FROM public.auditoria a
LEFT JOIN public.usuarios u ON a.id_usuario = u.id_usuario
WHERE a.modulo = 'Transacciones'
      AND a.operacion = 'RETIRO'
ORDER BY a.fecha DESC;

```

```
-- =====
```

```
-- CONSULTA 4: REPORTE DE TRANSACCIONES POR TIPO
```

```
-- Sirve para resumir las operaciones bancarias por categoría.
```

```
-- =====
```

```
-- Consulta normal
```

```

SELECT
    tipo_transaccion,
    COUNT(*) AS cantidad_operaciones,
    SUM(monto) AS monto_total,
    ROUND(AVG(monto), 2) AS monto_promedio
FROM public.transacciones
GROUP BY tipo_transaccion
ORDER BY monto_total DESC;

```

```
-- Índice para optimizar
```

```
CREATE INDEX IF NOT EXISTS idx_transacciones_fecha_tipo
```

```
ON public.transacciones (fecha_transaccion DESC, tipo_transaccion);
```

```
-- Análisis de rendimiento
```

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    tipo_transaccion,
```

```
    COUNT(*) AS cantidad_operaciones,
```

```
    SUM(monto) AS monto_total,
```

```
    ROUND(AVG(monto), 2) AS monto_promedio
```

```
FROM public.transacciones
```

```
GROUP BY tipo_transaccion
```

```
ORDER BY monto_total DESC;
```

```
-- =====
```

```
-- CONSULTA 5: CLIENTES CON MAYOR ACTIVIDAD FINANCIERA
```

```
-- Sirve para identificar qué clientes han realizado más
```

```
-- movimientos bancarios y cuánto dinero han movido.
```

```
-- =====
```

```
-- Consulta normal
```

```
SELECT
```

```
    cl.id_cliente,
```

```

    cl.dpi,
    cl.nombres,
    cl.apellidos,
    COUNT(t.id_transaccion) AS cantidad_transacciones,
    SUM(t.monto) AS monto_total_movido,
    MAX(t.fecha_transaccion) AS ultima_transaccion
FROM public.clientes cl
INNER JOIN public.cuentas c
    ON cl.id_cliente = c.id_cliente
INNER JOIN public.transacciones t
    ON c.id_cuenta = t.id_cuenta_origen
    OR c.id_cuenta = t.id_cuenta_destino
GROUP BY
    cl.id_cliente,
    cl.dpi,
    cl.nombres,
    cl.apellidos
ORDER BY
    monto_total_movido DESC,
    cantidad_transacciones DESC
LIMIT 3;

-- Índices para optimizar
CREATE INDEX IF NOT EXISTS idx_cuentas_cliente_movimiento
ON public.cuentas (id_cliente, id_cuenta);

```

```
CREATE INDEX IF NOT EXISTS idx_transacciones_origen_movimiento  
ON public.transacciones (id_cuenta_origen);
```

```
CREATE INDEX IF NOT EXISTS idx_transacciones_destino_movimiento  
ON public.transacciones (id_cuenta_destino);
```

```
-- Análisis de rendimiento
```

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    cl.id_cliente,
```

```
    cl.dpi,
```

```
    cl.nombres,
```

```
    cl.apellidos,
```

```
    COUNT(t.id_transaccion) AS cantidad_transacciones,
```

```
    SUM(t.monto) AS monto_total_movido,
```

```
    MAX(t.fecha_transaccion) AS ultima_transaccion
```

```
FROM public.clientes cl
```

```
INNER JOIN public.cuentas c
```

```
    ON cl.id_cliente = c.id_cliente
```

```
INNER JOIN public.transacciones t
```

```
    ON c.id_cuenta = t.id_cuenta_origen
```

```
    OR c.id_cuenta = t.id_cuenta_destino
```

```
GROUP BY
```

```
    cl.id_cliente,
```

```
    cl.dpi,
    cl.nombres,
    cl.apellidos
ORDER BY
    monto_total_movido DESC,
    cantidad_transacciones DESC
LIMIT 3;
```

```
-- =====
-- VERIFICACIÓN DE ÍNDICES CREADOS
-- Sirve para confirmar que los índices existen.
-- =====
```

```
SELECT
    tablename,
    indexname,
    indexdef
FROM pg_indexes
WHERE schemaname = 'public'
AND indexname IN (
    'idx_cuentas_numero_cuenta',
    'idx_transacciones_origen_fecha',
    'idx_transacciones_destino_fecha',
    'idx_clientes_dpi',
    'idx_cuentas_cliente_fecha',
```

```
'idx_auditoria_modulo_operacion_fecha',  
'idx_transacciones_fecha_tipo',  
'idx_cuentas_cliente_movimiento',  
'idx_transacciones_origen_movimiento',  
'idx_transacciones_destino_movimiento'  
)  
ORDER BY tablename, indexname;
```

Análisis comparativo de consultas optimizadas

- **Consulta 1: Historial de movimientos por cuenta**

Esta consulta permite revisar todas las transacciones en las que participa una cuenta bancaria, ya sea como cuenta origen o cuenta destino. Es útil para consultar el historial de movimientos de un cliente, verificar depósitos, retiros, transferencias y pagos asociados a una cuenta específica.

Antes de la optimización, PostgreSQL podía recorrer de forma más amplia las tablas transacciones y cuentas, especialmente porque la búsqueda se realiza por número de cuenta y luego se comparan las transacciones relacionadas como origen o destino. Esto puede volverse lento cuando existen muchas cuentas y muchos movimientos registrados.

Después de la optimización, se crearon índices sobre `numero_cuenta`, `id_cuenta_origen` e `id_cuenta_destino`. Con esto, PostgreSQL puede localizar más rápido la cuenta consultada y encontrar sus movimientos relacionados de forma más eficiente. Esta optimización beneficia directamente al módulo de historial de movimientos y al estado de cuenta del cliente.

Mejora obtenida: búsqueda más rápida de movimientos por cuenta, especialmente cuando la tabla de transacciones crezca.

- **Consulta 2: Estado de cuenta de un cliente**

Esta consulta muestra la información principal del cliente junto con sus cuentas bancarias, tipo de cuenta, saldo, estado y fecha de apertura. Es útil para generar un estado de cuenta básico o para que el cajero consulte rápidamente los productos financieros de un cliente.

Antes de la optimización, la consulta podía requerir una revisión más amplia de la tabla clientes para encontrar el DPI solicitado y luego buscar las cuentas asociadas. En una base con muchos clientes, buscar por DPI sin índice puede afectar el tiempo de respuesta.

Después de la optimización, se creó un índice sobre clientes.dpi y otro sobre cuentas.id_cliente junto con fecha_apertura. Esto permite encontrar al cliente de forma más rápida y luego recuperar sus cuentas ordenadas por fecha de apertura con mejor rendimiento.

Mejora obtenida: consulta más eficiente para atención al cliente y generación de estados de cuenta.

- **Consulta 3: Auditoría por operación**

Esta consulta permite revisar eventos específicos registrados en la tabla auditoria, filtrando por módulo, operación y fecha. Es importante para el rol auditor, ya que permite consultar acciones como retiros, transferencias, pagos, anulaciones, activaciones o inactivaciones.

Antes de la optimización, PostgreSQL podía recorrer gran parte de la tabla auditoria para encontrar los eventos solicitados. Esto puede afectar el rendimiento porque la auditoría suele crecer rápidamente, ya que registra muchas acciones del sistema.

Después de la optimización, se creó un índice compuesto sobre modulo, operacion y fecha. Este índice ayuda a localizar más rápido los registros de

auditoría relacionados con una operación específica y ordenarlos por fecha descendente.

Mejora obtenida: revisión más rápida de eventos críticos del sistema y mejor soporte para auditorías internas.

- **Consulta 4: Reporte de transacciones por tipo**

Esta consulta agrupa las transacciones por tipo de operación y calcula la cantidad de operaciones, el monto total y el monto promedio. Es útil para reportes administrativos y estadísticas generales del sistema bancario.

Antes de la optimización, PostgreSQL podía revisar todas las transacciones para agruparlas por tipo y calcular los totales. En una tabla pequeña esto no representa problema, pero con muchas operaciones bancarias puede aumentar el tiempo de consulta.

Después de la optimización, se creó un índice sobre `fecha_transaccion` y `tipo_transaccion`. Este índice ayuda especialmente cuando el reporte se filtra por fecha o cuando se generan reportes periódicos por tipo de operación.

Mejora obtenida: mejor rendimiento en reportes administrativos y análisis de operaciones bancarias.

- **Consulta 5: Clientes con mayor actividad financiera**

Esta consulta identifica los clientes con mayor movimiento bancario, calculando la cantidad de transacciones realizadas y el monto total movido. Además, ordena los resultados de mayor a menor actividad, permitiendo visualizar los clientes más activos del sistema.

Antes de la optimización, PostgreSQL podía recorrer ampliamente las tablas clientes, cuentas y transacciones, ya que debe relacionar cada cliente con sus

cuentas y luego buscar las transacciones donde esas cuentas participaron como origen o destino.

Después de la optimización, se crearon índices sobre cuentas.id_cliente, transacciones.id_cuenta_origen y transacciones.id_cuenta_destino. Esto permite relacionar más rápido los clientes con sus cuentas y encontrar con mayor eficiencia las transacciones asociadas.

Mejora obtenida: identificación más rápida de clientes con mayor actividad financiera, útil para análisis administrativo, monitoreo bancario y reportes gerenciales.

Resumen general del análisis comparativo

Consulta	Antes de optimizar	Después de optimizar	Beneficio principal
Historial de movimientos por cuenta	Revisión amplia de cuentas y transacciones.	Uso de índices por número de cuenta, origen y destino.	Consulta más rápida del historial bancario.
Estado de cuenta de cliente	Búsqueda general por DPI y cuentas asociadas.	Índices por DPI e ID de cliente.	Mejor atención y consulta de cuentas.
Auditoría por operación	Recorrido amplio de eventos de auditoría.	Índice por módulo, operación y fecha.	Auditoría más eficiente.
Reporte de transacciones por tipo	Agrupación sobre toda la tabla de transacciones.	Índice por fecha y tipo de transacción.	Reportes administrativos más rápidos.

Clientes con mayor actividad financiera	Relación amplia entre clientes, cuentas y transacciones.	Índices por cliente, cuenta origen y cuenta destino.	Mejor análisis de clientes activos.
--	--	--	-------------------------------------

En general, las consultas fueron optimizadas mediante índices en columnas utilizadas frecuentemente en filtros, uniones y ordenamientos. Aunque en una base de datos pequeña PostgreSQL puede seguir usando lectura secuencial, los índices preparan el sistema para mantener buen rendimiento cuando aumente el volumen de clientes, cuentas, transacciones y registros de auditoría.

Seguridad del sistema

La seguridad del sistema GBA CREDOMATIC se basa en el control de acceso por roles, validación de credenciales, manejo de sesiones, restricción de módulos y protección de operaciones críticas.

El sistema utiliza autenticación mediante usuario o correo y contraseña. Las contraseñas se almacenan de forma cifrada, evitando guardar claves en texto plano dentro de la base de datos.

Control de acceso por roles

- 1) El sistema cuenta con tres roles principales: administrador, cajero y auditor.
Cada rol tiene acceso únicamente a las funciones que le corresponden.
- 2) El administrador puede gestionar usuarios, clientes, cuentas, reportes y estados del sistema.
- 3) El cajero puede registrar clientes, crear cuentas, procesar transacciones, registrar pagos y anular operaciones permitidas.
- 4) El auditor puede consultar eventos, transacciones y registros de auditoría.

- 5) Este control evita que un usuario realice acciones que no corresponden a sus responsabilidades.

Validación de usuarios activos

El sistema verifica que el usuario exista, que la contraseña sea correcta y que el estado del usuario sea ACTIVO. Si un usuario se encuentra inactivo, no puede iniciar sesión.

Esta validación permite que el administrador pueda bloquear el acceso de usuarios internos cuando sea necesario.

Protección de operaciones críticas

Algunas acciones administrativas, como activar o inactivar usuarios, clientes o cuentas, requieren confirmación mediante contraseña del administrador. Esto agrega una capa adicional de seguridad en operaciones sensibles.

Auditoría de acciones

Las acciones importantes del sistema quedan registradas en la tabla de auditoría. Esto permite mantener trazabilidad y revisar posteriormente qué usuario realizó determinada operación.

Seguridad de la base de datos

PostgreSQL mantiene relaciones mediante llaves primarias y foráneas, restricciones de integridad y validaciones sobre los datos. Estas características ayudan a evitar registros inconsistentes y operaciones inválidas.

Manual de usuario

El sistema GBA CREDOMATIC es una aplicación bancaria web diseñada para administrar procesos básicos de una entidad financiera. El sistema funciona mediante una interfaz web, por lo que el usuario puede acceder desde un navegador. Para utilizarlo correctamente, es necesario contar con credenciales válidas, conexión al sistema y un rol activo dentro de la base de datos.

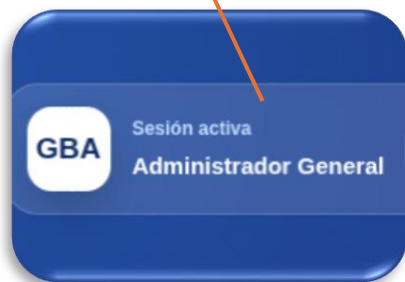
Objetivo

Orientar al usuario en el uso de las principales funciones del sistema bancario GBA CREDOMATIC, explicando el acceso al sistema, la navegación por los módulos y el uso de las opciones disponibles según el rol asignado.

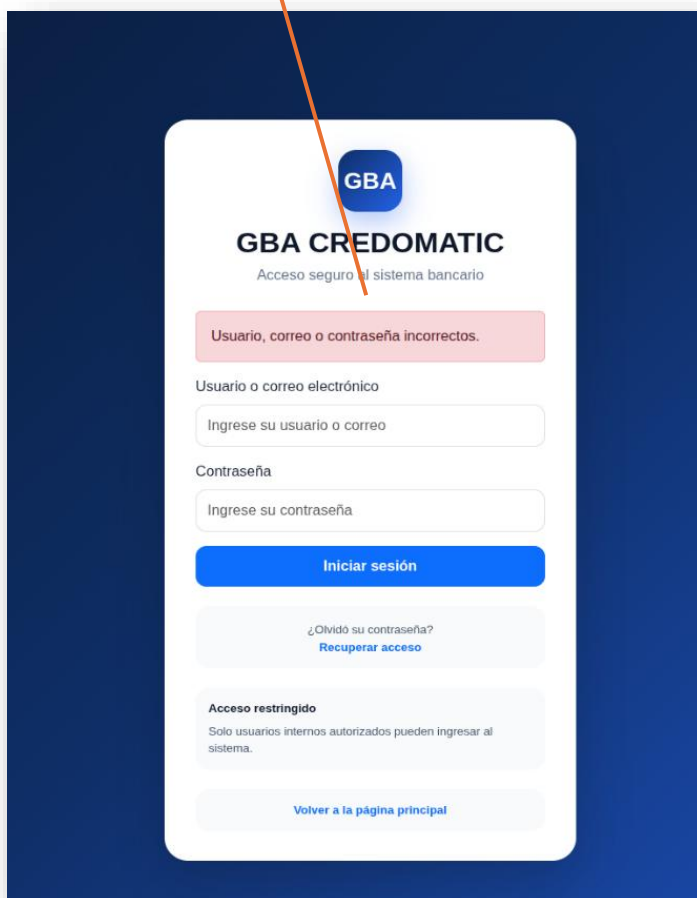
Acceso al sistema

El usuario ingresa al sistema desde el navegador web. En la pantalla de inicio de sesión debe colocar su usuario o correo y su contraseña.

Si los datos son correctos, el sistema valida el rol del usuario y lo redirige al panel correspondiente.

Una interfaz de usuario para el inicio de sesión. El encabezado muestra el logo 'GBA' en un círculo azul, seguido de 'GBA CREDOMATIC' en negrita y 'Acceso seguro al sistema bancario' en un tamaño de fuente menor. Hay dos campos de entrada: 'Usuario o correo electrónico' con el valor 'admin_banco' y 'Contraseña' con caracteres ocultos por asteriscos. Debajo de los campos hay un botón azul que dice 'Procesando...' con un icono de carga. Más abajo, un enlace azul dice '¿Olvidó su contraseña? Recuperar acceso'. En la parte inferior, un recuadro gris indica 'Acceso restringido' con el texto 'Solo usuarios internos autorizados pueden ingresar al sistema.' y un enlace azul 'Volver a la página principal'. Una flecha naranja curva se extiende desde el campo de contraseña hacia el párrafo 'El usuario ingresa al sistema...' del texto anterior.

Si las credenciales son incorrectas o el usuario se encuentra inactivo, el sistema muestra un mensaje de error.



The image shows a login interface for 'GBA CREDOMATIC'. At the top, there is a blue square logo with 'GBA' in white. Below it, the text 'GBA CREDOMATIC' is displayed in bold, followed by the subtitle 'Acceso seguro al sistema bancario'. A red error message box states: 'Usuario, correo o contraseña incorrectos.' Below this, there are two input fields: 'Usuario o correo electrónico' with the placeholder 'Ingrese su usuario o correo', and 'Contraseña' with the placeholder 'Ingrese su contraseña'. A blue button labeled 'Iniciar sesión' is positioned below the password field. Underneath the button, a link reads '¿Olvidó su contraseña? Recuperar acceso'. Further down, a section titled 'Acceso restringido' explains: 'Solo usuarios internos autorizados pueden ingresar al sistema.' At the bottom, a link says 'Volver a la página principal'. An orange arrow points from the text above to the error message box.

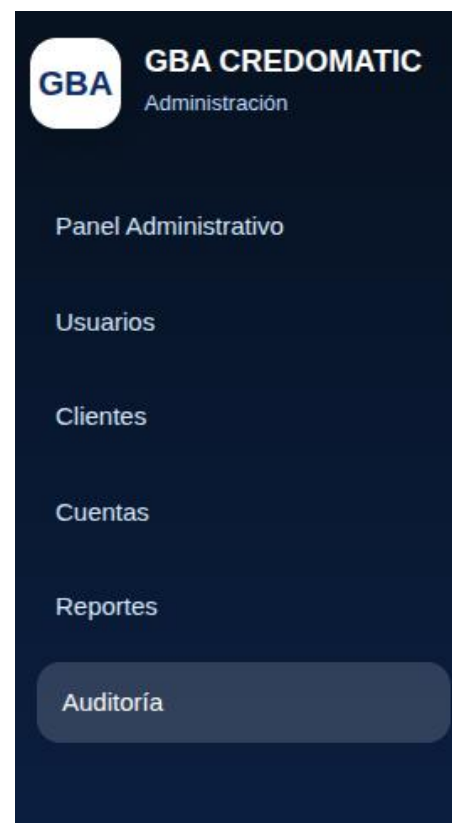
Rol administrador

El panel del administrador permite gestionar usuarios internos, consultar información general del sistema, revisar estadísticas y administrar estados de usuarios, clientes y cuentas.

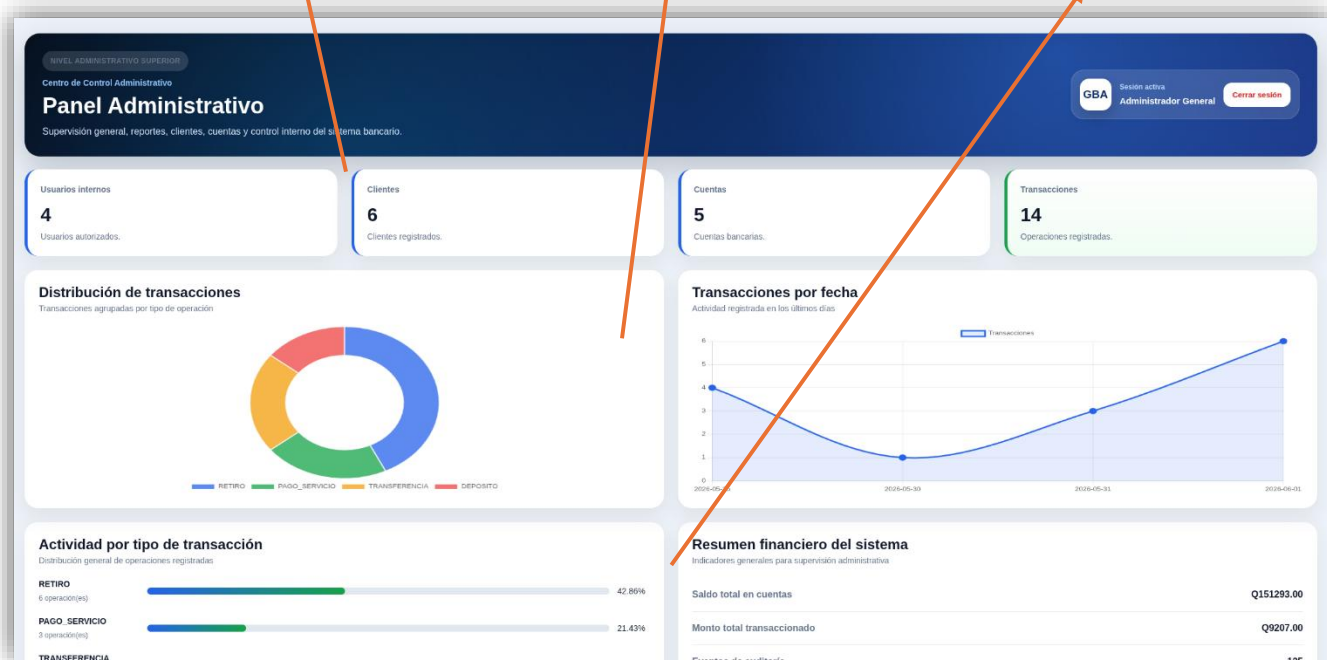
Al ingresar como administrador, el sistema muestra un panel con información resumida, como cantidad de usuarios, clientes, cuentas, transacciones, eventos de auditoría, saldo y monto totales de transacciones.

Funciones principales del administrador:

- 1) Crear usuarios internos.
- 2) Consultar usuarios registrados.
- 3) Activar o inactivar usuarios.
- 4) Activar o inactivar clientes.
- 5) Activar o inactivar cuentas.
- 6) Consultar reportes y estadísticas.



Panel administrativo general donde se ven el total de usuarios, clientes, cuentas y transacciones. Estadísticas del flujo de transacciones. Apartado de actividad.



Apartado para la creación de usuarios registrando su nombres y apellidos, asignándole un correo y rol correspondiente. Opción para activar/inactivar usuarios

Usuarios
Gestión de usuarios internos y roles autorizados.

Registrar usuario interno
Creación de usuarios administrativos, cajeros o auditores sin salir del panel

Nombre completo: Ejemplo: Juan Pérez

Usuario: Ejemplo: jperez

Correo electrónico: usuario@correo.com

Rol: Seleccione un rol

Contraseña: Mínimo 6 caracteres

Confirmar contraseña: Repita la contraseña

Guardar usuario interno

Usuarios internos del sistema
Administración de accesos autorizados por rol

Usuario	Nombre completo	Correo	Rol	Estado	Fecha registro	Acción
ErAdmin_banco	Erick Fernando Sanchez Monte Blanco	eradmin060@glacredomatic.com.gt	ADMINISTRADOR	ACTIVO	2026-05-30 17:20:10.164499	Contraseña admin Inactivar
auditor_banco	Auditor interno	auditor@glacredomatic.com.gt	AUDITOR	ACTIVO	2026-05-24 23:07:15.864126	Contraseña admin Inactivar
cajero_banco	Cajero Bancario	cajero@glacredomatic.com.gt	CAJERO	ACTIVO	2026-05-24 23:07:15.864126	Contraseña admin Inactivar
admin_banco	Administrador General	admin@glacredomatic.com.gt	ADMINISTRADOR	ACTIVO	2026-05-24 23:07:15.864126	Usuario actual

Apartado para visualizar las cuentas bancarias registradas. Opcion para filtrar cuenta.

NIVEL ADMINISTRATIVO SUPERIOR
Centro de Control Administrativo
Cuentas
Consulta administrativa de cuentas bancarias asociadas a clientes registrados.
GBA Sesión activa Administrador General Cerrar sesión

Cuentas bancarias
Consulta administrativa de cuentas asociadas a clientes registrados

Número de cuenta	Cliente	DPI	Tipo	Saldo	Estado	Fecha apertura	Acción
1000010005	Juan amoldo Escamilla Perez	78654326789057	EMPRESARIAL	Q99943.00	ACTIVA	2026-05-31 21:19:55.875282	Contraseña admin Inactivar
1000010004	Luis Gustavo Martinez Arlaza	78654890273817	AHORRO	Q7150.00	ACTIVA	2026-05-30 07:35:04.536273	Contraseña admin Inactivar
1000010003	Jose Ramon Alvarado perez	89584789398282	EMPRESARIAL	Q43650.00	ACTIVA	2026-05-29 03:11:46.786356	Contraseña admin Inactivar
1000010002	Pedro Arnoldo Infante Marruecos	9483756383902	MONETARIA	Q0.00	ACTIVA	2026-05-25 02:32:08.831346	Contraseña admin Inactivar
1000010001	Juan Manolo Perez Palacios	9098778790998	AHORRO	Q550.00	ACTIVA	2026-05-25 01:37:41.518446	Contraseña admin Inactivar

Apartado para visualizar los clientes registrados. Opcion para filtrar clientes.

NIVEL ADMINISTRATIVO SUPERIOR
Centro de Control Administrativo
Clientes
Consulta administrativa de clientes registrados por el área operativa.
GBA Sesión activa Administrador General Cerrar sesión

Clientes registrados
Consulta administrativa de clientes creados por el área operativa

DPI	Nombre completo	Teléfono	Correo	Dirección	Estado	Acción
78654326789057	Juan amoldo Escamilla Perez	6787346	juana67@gmail.com	kilometro 200 colonia cerro alegre	ACTIVO	Contraseña admin Inactivar
09856142356718	Mario Rolando Castañeda Pineda	7654783	Mario99@gmail.com	kilometro 111 avenida siempre la chona	ACTIVO	Contraseña admin Inactivar
78654890273817	Luis Gustavo Martinez Arlaza	5674324	Luisg78@gmail.com	kilometro 311 avenida colonia la esperanza	ACTIVO	Contraseña admin Inactivar
89584789398282	Jose Ramon Alvarado perez	677675454	josep777@gmail.com	kilometro 888 avenida siempre viva 67	ACTIVO	Contraseña admin Inactivar
9483756383902	Pedro Arnoldo Infante Marruecos	7847875	pedron56@gmail.com	kilometro 777 avenida la abeja	ACTIVO	Contraseña admin Inactivar
9098778790998	Juan Manolo Perez Palacios	1039840	juarp123@gmail.com	kilometro 7000 fd mexico	ACTIVO	Contraseña admin Inactivar

Apartado para visualizar el reporte de transacciones como depósitos, retiros etc.

NIVEL ADMINISTRATIVO SUPERIOR

Centro de Control Administrativo

Reportes

Consulta administrativa de transacciones y actividad financiera.

GBA

Sesión activa

Administrador General

Cerrar sesión

Reporte de transacciones

Supervisión administrativa de depósitos, retiros, transferencias, pagos y anulaciones

Mostrar filtros

Total de transacciones

14

Monto total

Q9207.00

Eventos auditados

143

Código	Tipo	Cuenta origen	Cuenta destino	Monito	Estado	Fecha	Descripción
TRX-20260601010637050456	PAGO_SERVICIO	5	N/A	Q77.00	COMPLETADA	2026-06-01 01:06:37.070696	Pago de servicio con referencia 8765439087651.
TRX-20260601010546988622	RETIRO	2	N/A	Q500.00	COMPLETADA	2026-06-01 01:05:46.998020	Retiro realizado en ventanilla.
TRX-20260601010535888663	RETIRO	5	N/A	Q300.00	COMPLETADA	2026-06-01 01:05:35.896397	Retiro realizado en ventanilla.
TRX-20260601010457374075	TRANSFERENCIA	5	2	Q100.00	COMPLETADA	2026-06-01 01:04:57.384041	Transferencia realizada en ventanilla.
TRX-20260601010435466132	DEPOSITO	N/A	4	Q200.00	COMPLETADA	2026-06-01 01:04:35.479462	Depósito realizado en ventanilla.
TRX-20260601010350943472	DEPOSITO	N/A	4	Q600.00	COMPLETADA	2026-06-01 01:03:50.954652	Depósito realizado en ventanilla.
TRX-20260531232432979225	RETIRO	5	N/A	Q500.00	COMPLETADA	2026-05-31 23:24:32.988182	Retiro realizado en ventanilla.
TRX-20260531212148838545	PAGO_SERVICIO	5	N/A	Q80.00	COMPLETADA	2026-05-31 21:21:48.848329	Pago de servicio con referencia 6754678976543.
TRX-20260531212030201666	TRANSFERENCIA	3	5	Q1000.00	COMPLETADA	2026-05-31 21:20:30.212437	Transferencia realizada en ventanilla.
TRX-20260530142427227397	TRANSFERENCIA	3	2	Q350.00	COMPLETADA	2026-05-30 14:24:27.237970	Transferencia realizada en ventanilla.
TRX-20260525031214098755	RETIRO	3	N/A	Q5000.00	COMPLETADA	2026-05-25 03:12:14.108992	Retiro realizado en ventanilla.
TRX-20260525023330146531	PAGO_SERVICIO	1	N/A	Q250.00	COMPLETADA	2026-05-25 02:33:30.157597	Pago de servicio con referencia 4354565757657.
TRX-20260525023241738503	RETIRO	2	N/A	Q50.00	COMPLETADA	2026-05-25 02:32:41.747847	Retiro realizado en ventanilla.
TRX-20260525013806430225	RETIRO	1	N/A	Q200.00	COMPLETADA	2026-05-25 01:38:06.439945	Retiro realizado en ventanilla.

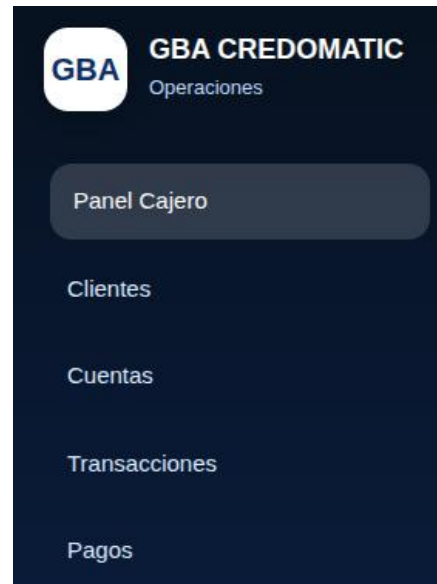
Tabla para visualizar cada transacción con sus datos correspondientes, que tipo de transacción se realizó, su estado, la fecha y su descripción.

Rol cajero

El cajero puede realizar operaciones relacionadas con clientes, cuentas y movimientos bancarios.

Funciones principales del cajero:

- 1) Registrar clientes.
- 2) Crear cuentas bancarias.
- 3) Procesar depósitos.
- 4) Procesar retiros.
- 5) Realizar transferencias entre cuentas.
- 6) Registrar pagos de servicios.
- 7) Anular transacciones permitidas.
- 8) Consultar transacciones recientes.



Panel principal para llevar el control de clientes, cuentas y transacciones registradas.

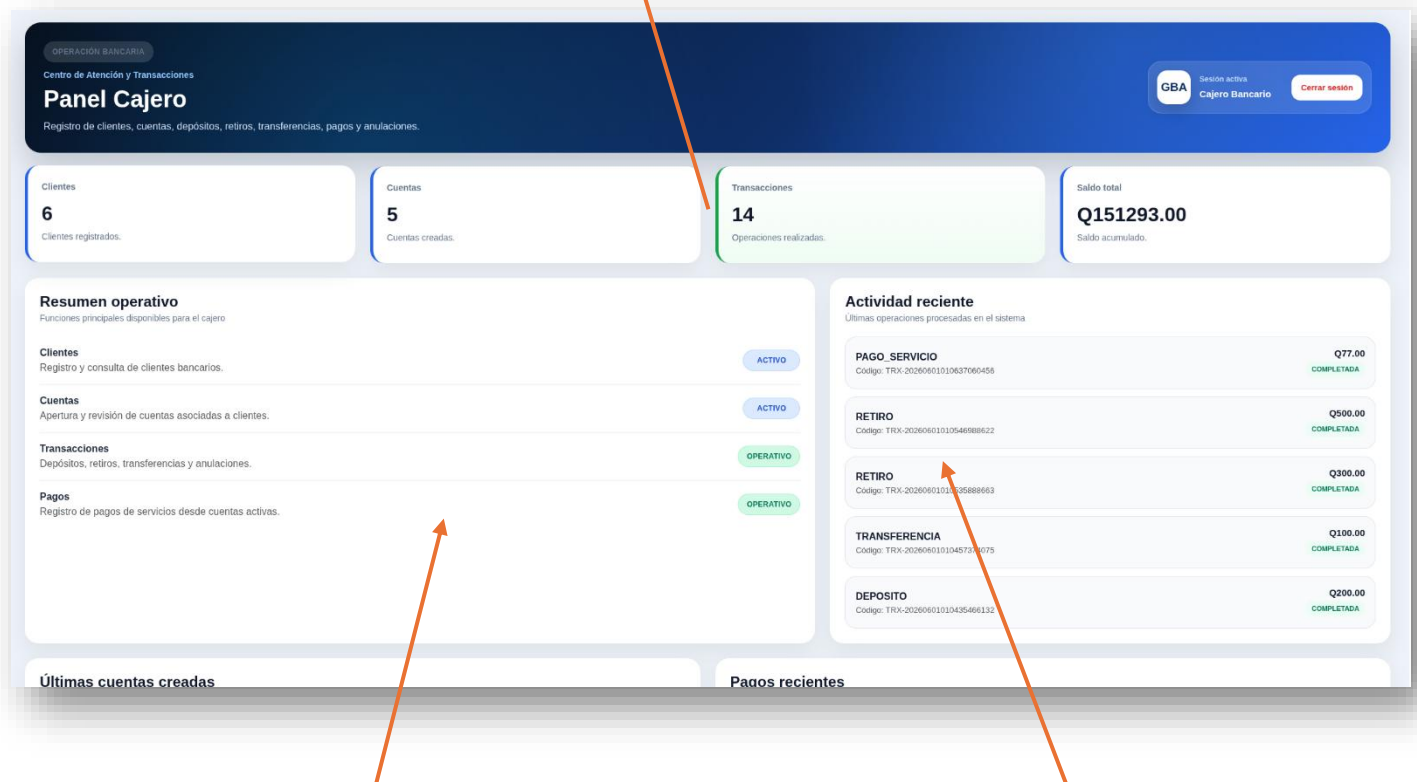


Tabla mostrando el estado de las funciones.

Tablas de control de actividad.

Apartado para el registro de clientes.

Tabla de clientes registrados y su estado.

OPERACIÓN BANCARIA

Centro de Atención y Transacciones

Cientes

Registro y consulta de clientes bancarios.

GBA

Sesión activa
Cajero Bancario

Cerrar sesión

Registrar cliente

Ingreso de datos básicos del cliente bancario

DPI

Nombres

Apellidos

Teléfono

Correo

Dirección

Guardar cliente

Cientes registrados

Listado general de clientes creados

DPI	Nombre completo	Teléfono	Correo	Dirección	Estado
78654326789057	Juan arnoldo Escamilla Perez	6787346	juanar67@gmail.com	kilometro 200 colonia cerro alegre	Activo
98056142356718	Mario Rolando Castañeda Pineda	7654793	Mario99@gmail.com	kilometro 111 avenida siempre la chona	Activo
78654890273817	Luis Gustavo Martinez Arriaza	5674324	Luisg76@gmail.com	kilometro 311 avenida colonia la esperanza	Activo
89584789398282	Jose Ramon Alvarado perez	677675454	josep777@gmail.com	kilometro 888 avenida siempreviva 67	Activo
9483756383902	Pedro Arnoldo Infante Marruecos	7647675	pedrois56@gmail.com	kilometro 777 avenida la abeja	Activo
9098778790998	Juan Manolo Perez Palacios	1039840	juamp123@gmail.com	kilometro 7000 fd mexico	Activo

Apartado de creacion de cuentas.

Tabla de cuentas registradas y su estado.

OPERACION BANCARIA

Centro de Atención y Transacciones

Cuentas

Apertura y consulta de cuentas bancarias.

GBA

Sesión activa
Cajero Bancario

Cerrar sesión

Apertura de cuenta

Creación de cuenta bancaria asociada a un cliente

Cliente

Seleccione un cliente

Crear cuenta

Tipo de cuenta

Seleccione tipo

Seleccione tipo

AHORRO

EMPRESARIAL

MONETARIA

Saldo inicial

0

Cuentas registradas

Lista de cuentas bancarias activas o registradas

Número de cuenta	Cliente	Tipo	Saldo	Estado
1000010005	Juan amoldo Escanilla Perez	EMPRESARIAL	Q99943.00	ACTIVA
1000010004	Luis Gustavo Martinez Arriaza	AHORRO	Q7150.00	ACTIVA
1000010003	Jose Ramon Alvarado perez	EMPRESARIAL	Q43650.00	ACTIVA
1000010002	Pedro Arroldo Infante Manrueros	MONETARIA	Q0.00	ACTIVA
1000010001	Juan Manolo Perez Palacios	AHORRO	Q550.00	ACTIVA

Apartado para realizar todo tipo de transacciones. Apartado para anular transacciones.

OPERACIÓN BANCARIA

Centro de Atención y Transacciones

Transacciones

Depósitos, retiros, transferencias y anulación de operaciones.

GBA Sesión activa
Cajero Bancario Cerrar sesión

Procesar transacción

Depósitos, retiros y transferencias entre cuentas

Tipo de transacción

Seleccione

Depósito

Retiro

Transferencia

Cuenta origen

No aplica

Cuenta destino

No aplica

Monto

Anulación de transacción

Permite revertir una transacción completada y actualizar el saldo correspondiente

Transacción a anular

Seleccione una transacción completada

Anular transacción

Últimas transacciones

Historial reciente de movimientos registrados

Código	Tipo	Origen	Destino	Monto	Estado	Fecha
TRX-20260601010637060456	PAGO_SERVICIO	5	N/A	Q77.00	COMPLETADA	2026-06-01 01:06:37.070696
TRX-20260601010546988622	RETIRO	2	N/A	Q500.00	COMPLETADA	2026-06-01 01:05:46.998020
TRX-20260601010535888663	RETIRO	5	N/A	Q300.00	COMPLETADA	2026-06-01 01:05:35.896397
TRX-20260601010457374079	TRANSFERENCIA	5	2	Q100.00	COMPLETADA	2026-06-01 01:04:57.384041
TRX-20260601010435466132	DEPOSITO	N/A	4	Q200.00	COMPLETADA	2026-06-01 01:04:35.479462

Apartado para realizar pagos de de servicios. Tabla que registra los pagos recientes.

OPERACIÓN BANCARIA

Centro de Atención y Transacciones

Pagos

Pagos de servicios realizados desde cuentas.

GBA Sesión activa
Cajero Bancario Cerrar sesión

Pagos de servicio

Pago de servicios asociados a una cuenta bancaria

Cuenta

Seleccione cuenta

Servicio

Seleccione servicio

Seleccione servicio

AGUA POTABLE

ENERGIA ELECTRICA

INTERNET

TELEFONIA

Referencia

Monto

Registrar pago

Pagos registrados

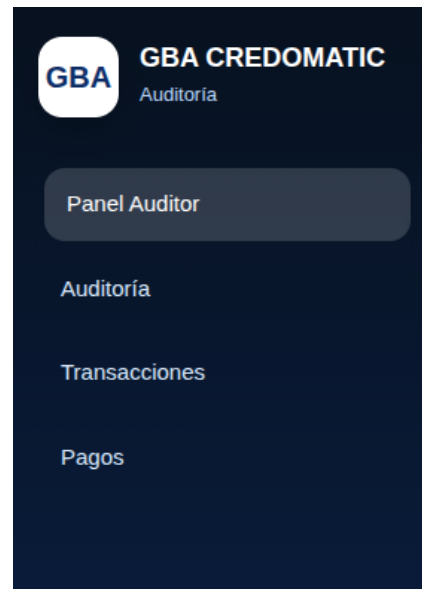
Historial reciente de pagos de servicios

Código	Cuenta	Servicio	Referencia	Monto	Estado	Fecha
PAG-20260601010637060436	1000010005	AGUA POTABLE	8765439097651	Q77.00	PAGADO	2026-06-01 01:06:37.070696
PAG-20260531212148838529	1000010005	TELEFONIA	6754678976543	Q80.00	PAGADO	2026-05-31 21:21:48.948329
PAG-20260525023330146513	1000010001	INTERNET	4354565757657	Q250.00	PAGADO	2026-05-25 02:33:30.157597

Rol auditor

El auditor puede revisar información relacionada con eventos del sistema y operaciones realizadas.

- 1) Funciones principales del auditor:
- 2) Consultar registros de auditoría.
- 3) Revisar transacciones realizadas.
- 4) Revisar pagos registrados.
- 5) Verificar acciones ejecutadas por usuarios internos.



Panel principal para llevar el control de los eventos del sistema bancario GBA

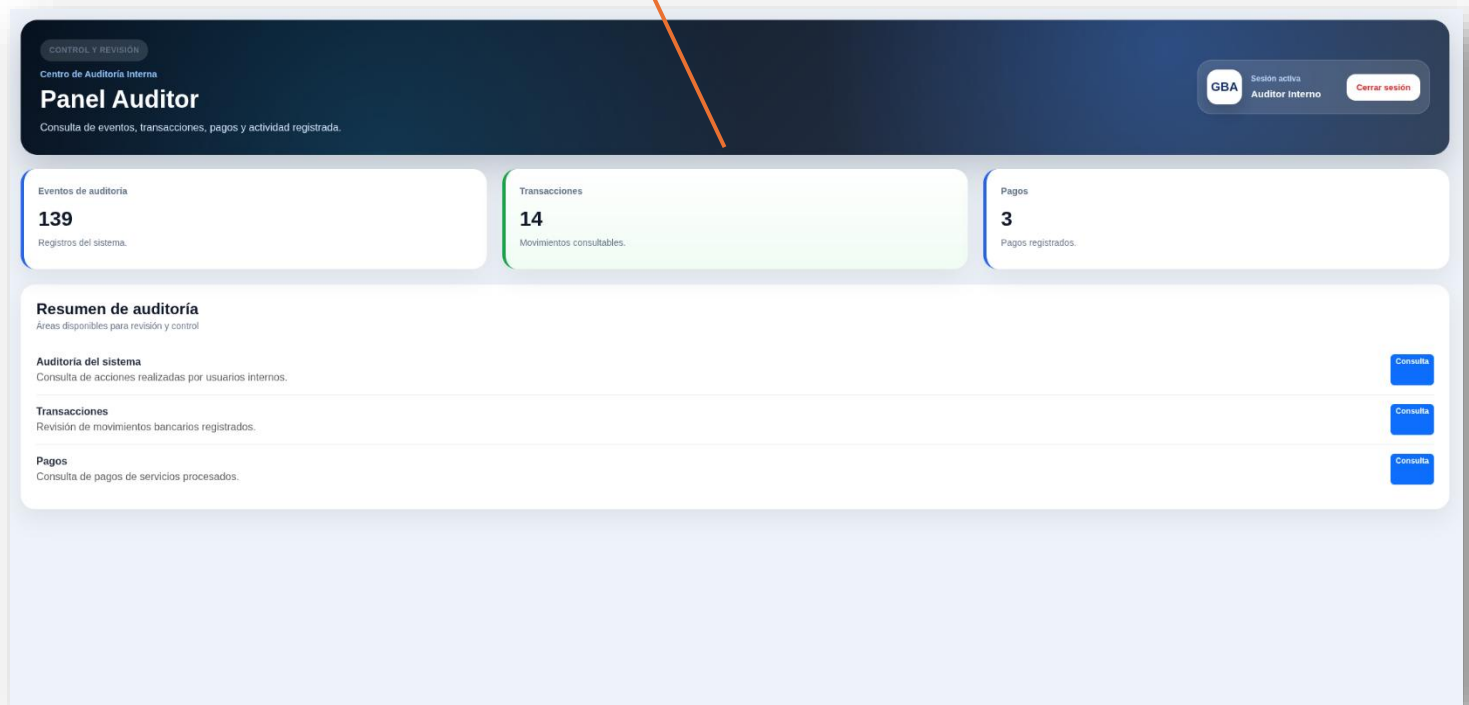


Tabla que registrar todos los movimientos del sistema bancario

CONTROL Y REVISIÓN				
Centro de Auditoría Interna				
Auditoría				
Eventos registrados dentro del sistema.				
Auditoría del sistema				
Eventos registrados dentro del sistema bancario				
Fecha	Usuario	Módulo	Operación	Descripción
2026-06-02 19:31:20.835011	auditor_banco	Seguridad	LOGIN	El usuario auditor_banco inició sesión con rol AUDITOR.
2026-06-02 19:31:10.686090	cajero_banco	Seguridad	LOGOUT	El usuario cajero_banco cerró sesión.
2026-06-02 19:18:41.533989	cajero_banco	Seguridad	LOGIN	El usuario cajero_banco inició sesión con rol CAJERO.
2026-06-02 19:18:29.237974	admin_banco	Seguridad	LOGOUT	El usuario admin_banco cerró sesión.
2026-06-02 17:50:39.910185	admin_banco	Seguridad	LOGIN	El usuario admin_banco inició sesión con rol ADMINISTRADOR.
2026-06-02 17:50:26.988033	auditor_banco	Seguridad	LOGOUT	El usuario auditor_banco cerró sesión.
2026-06-02 17:48:00.416147	auditor_banco	Seguridad	LOGIN	El usuario auditor_banco inició sesión con rol AUDITOR.
2026-06-02 17:48:00.657596	auditor_banco	Seguridad	LOGOUT	El usuario auditor_banco cerró sesión.
2026-06-02 17:41:22.309189	auditor_banco	Seguridad	LOGIN	El usuario auditor_banco inició sesión con rol AUDITOR.
2026-06-02 17:41:07.809597	cajero_banco	Seguridad	LOGOUT	El usuario cajero_banco cerró sesión.
2026-06-02 17:38:16.107455	cajero_banco	Seguridad	LOGIN	El usuario cajero_banco inició sesión con rol CAJERO.
2026-06-02 17:37:54.465193	admin_banco	Seguridad	LOGOUT	El usuario admin_banco cerró sesión.
2026-06-02 17:27:03.994574	admin_banco	Seguridad	LOGIN	El usuario admin_banco inició sesión con rol ADMINISTRADOR.
2026-06-02 17:26:49.814330	admin_banco	Seguridad	LOGOUT	El usuario admin_banco cerró sesión.
2026-06-02 17:26:45.190759	admin_banco	Seguridad	LOGIN	El usuario admin_banco inició sesión con rol ADMINISTRADOR.
2026-06-02 17:26:33.517431	admin_banco	Seguridad	LOGOUT	El usuario admin_banco cerró sesión.
2026-06-02 17:26:21.590218	admin_banco	Seguridad	LOGIN	El usuario admin_banco inició sesión con rol ADMINISTRADOR.
2026-06-02 17:26:14.510699	admin_banco	Seguridad	LOGOUT	El usuario admin_banco cerró sesión.
2026-06-02 17:26:10.555843	admin_banco	Seguridad	LOGIN	El usuario admin_banco inició sesión con rol ADMINISTRADOR.

Tabla que registra todas las transacciones realizadas, retiros, transacciones etc.

CONTROL Y REVISIÓN							
Centro de Auditoría Interna							
Transacciones							
Consulta de movimientos bancarios registrados.							
Transacciones registradas							
Consulta de movimientos bancarios procesados							
Código	Tipo	Cuenta origen	Cuenta destino	Monto	Estado	Fecha	Descripción
TRX-20260601010637060456	PAGO_SERVICIO	5	N/A	Q77.00	COMPLETADA	2026-06-01 01:06:37.070696	Pago de servicio con referencia 8765439087851.
TRX-20260601010546988622	RETIRO	2	N/A	Q500.00	COMPLETADA	2026-06-01 01:05:46.998020	Retiro realizado en ventanilla.
TRX-20260601010535888663	RETIRO	5	N/A	Q300.00	COMPLETADA	2026-06-01 01:05:35.896397	Retiro realizado en ventanilla.
TRX-20260601010457374075	TRANSFERENCIA	5	2	Q100.00	COMPLETADA	2026-06-01 01:04:57.384041	Transferencia realizada en ventanilla.
TRX-20260601010435466132	DEPOSITO	N/A	4	Q200.00	COMPLETADA	2026-06-01 01:04:35.479462	Depósito realizado en ventanilla.
TRX-202606010103509643472	DEPOSITO	N/A	4	Q600.00	COMPLETADA	2026-06-01 01:03:50.954652	Depósito realizado en ventanilla.
TRX-20260531232432979225	RETIRO	5	N/A	Q500.00	COMPLETADA	2026-05-31 23:24:32.988182	Retiro realizado en ventanilla.
TRX-20260531212148839545	PAGO_SERVICIO	5	N/A	Q80.00	COMPLETADA	2026-05-31 21:21:48.848329	Pago de servicio con referencia 6754678978543.
TRX-20260531212030201666	TRANSFERENCIA	3	5	Q1000.00	COMPLETADA	2026-05-31 21:20:30.212437	Transferencia realizada en ventanilla.
TRX-20260530142427227970	TRANSFERENCIA	3	2	Q350.00	COMPLETADA	2026-05-30 14:24:27.237970	Transferencia realizada en ventanilla.
TRX-20260525031214098785	RETIRO	3	N/A	Q5000.00	COMPLETADA	2026-05-25 03:12:14.108892	Retiro realizado en ventanilla.
TRX-20260525023330149531	PAGO_SERVICIO	1	N/A	Q250.00	COMPLETADA	2026-05-25 02:33:30.137597	Pago de servicio con referencia 4354565757857.
TRX-20260525023241738503	RETIRO	2	N/A	Q50.00	COMPLETADA	2026-05-25 02:32:41.747847	Retiro realizado en ventanilla.
TRX-20260525013806430225	RETIRO	1	N/A	Q300.00	COMPLETADA	2026-05-25 01:38:06.439645	Retiro realizado en ventanilla.

Tabla que registrar todos los pagos realizados con su código correspondiente y el estado en el que se encuentra.

CONTROL Y REVISIÓN

Centro de Auditoría Interna

Pagos

Consulta de pagos de servicios registrados.

GBA

Sesión activa

Auditor Interno

Cerrar sesión

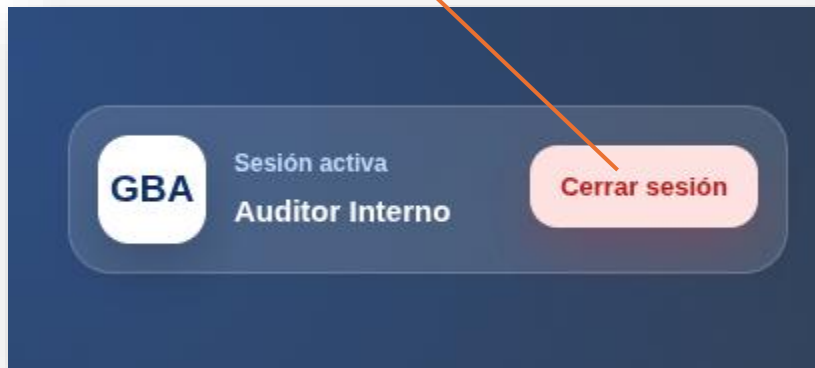
Pagos registrados

Consulta de pagos de servicios realizados

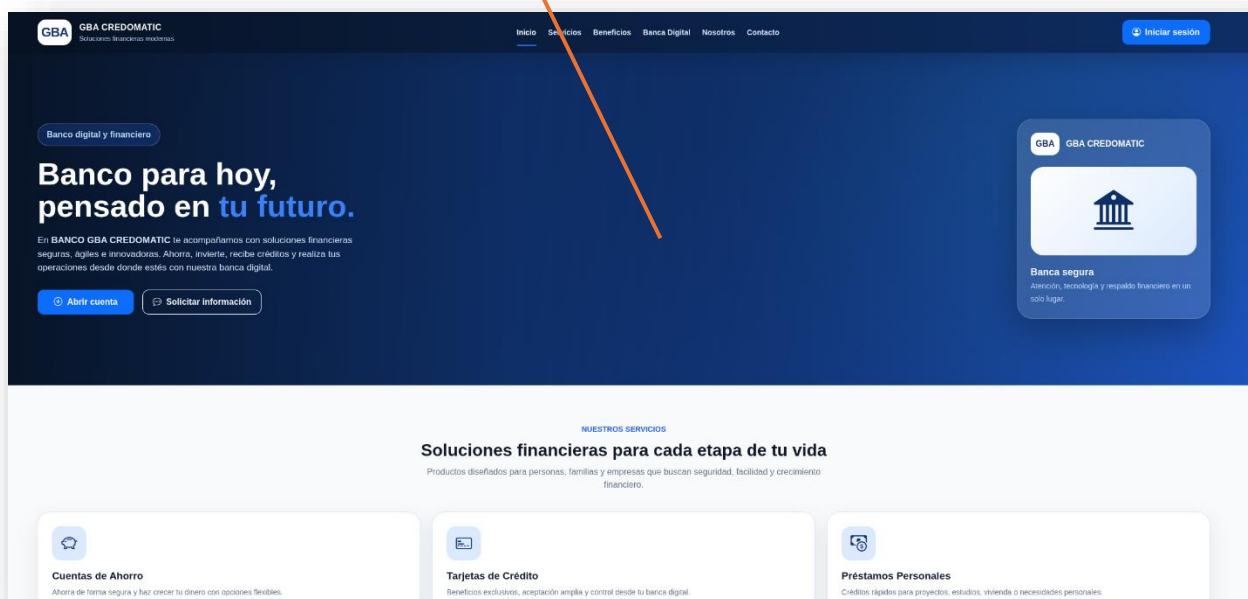
Código	Cuenta	Servicio	Referencia	Monto	Estado	Fecha
PAG-20260601010637060436	1000010005	AGUA POTABLE	6765439087651	Q77.00	PAGADO	2026-06-01 01:06:37.070696
PAG-20260531212148839529	1000010005	TELEFONIA	6754678970543	Q80.00	PAGADO	2026-05-31 21:21:48.848329
PAG-20260525023330146513	1000010001	INTERNET	4354565757657	Q250.00	PAGADO	2026-05-25 02:33:30.157597

Cierre de sesión

El usuario puede cerrar sesión desde el sistema dando clic al botón cerrar sesion. Al cerrar sesión, se elimina la información de sesión activa.



Luego de cerrar sesión el sistema regresa a la pantalla de inicio.



Reporte de pruebas

El reporte de pruebas permite comprobar que las funcionalidades principales del sistema fueron ejecutadas correctamente. Estas pruebas validan el comportamiento esperado en módulos como inicio de sesión, usuarios, clientes, cuentas, transacciones, pagos, auditoría, respaldo, recuperación y consultas optimizadas.

Prueba de inicio de sesión

- **Objetivo:** Verificar que un usuario registrado pueda iniciar sesión correctamente.
- **Resultado esperado:** El sistema debe validar las credenciales y redirigir al panel correspondiente según el rol.
- **Resultado obtenido:** El usuario accedió correctamente al sistema y fue redirigido a su panel.

Prueba de usuario inactivo

- **Objetivo:** Comprobar que un usuario inactivo no pueda iniciar sesión.
- **Resultado esperado:** El sistema debe mostrar un mensaje indicando que el usuario se encuentra inactivo.
- **Resultado obtenido:** El sistema bloqueó el acceso del usuario inactivo.

Prueba de registro de cliente

- **Objetivo:** Verificar que el cajero pueda registrar un cliente nuevo.
- **Resultado esperado:** El sistema debe guardar los datos del cliente y mostrarlo en la tabla correspondiente.
- **Resultado obtenido:** El cliente fue registrado correctamente.

Prueba de creación de cuenta

- Objetivo: Comprobar que el cajero pueda crear una cuenta bancaria para un cliente.
- Resultado esperado: El sistema debe generar la cuenta, asociarla al cliente y registrar el saldo inicial.
- Resultado obtenido: La cuenta fue creada correctamente y quedó asociada al cliente.

Prueba de depósito

- Objetivo: Verificar que el sistema permita incrementar el saldo de una cuenta destino.
- Resultado esperado: El saldo de la cuenta debe aumentar y debe registrarse la transacción.
- Resultado obtenido: El depósito fue procesado correctamente.

Prueba de retiro

- Objetivo: Validar que el sistema permita retirar dinero únicamente si existen fondos suficientes.
- Resultado esperado: Si el saldo es suficiente, el sistema descuenta el monto y registra la transacción.
- Resultado obtenido: El retiro fue procesado correctamente cuando existían fondos suficientes.

Prueba de fondos insuficientes

- Objetivo: Comprobar que el sistema no permita retirar o transferir un monto mayor al saldo disponible.
- Resultado esperado: El sistema debe mostrar un mensaje de error y no debe modificar el saldo.
- Resultado obtenido: El sistema bloqueó la operación por fondos insuficientes.

Prueba de transferencia

- Objetivo: Validar que el sistema descuente dinero de una cuenta origen y lo acredite a una cuenta destino.
- Resultado esperado: Ambos saldos deben actualizarse correctamente y la transacción debe quedar registrada.
- Resultado obtenido: La transferencia se realizó correctamente.

Prueba de pago de servicio

- Objetivo: Verificar que el sistema permita pagar un servicio desde una cuenta bancaria.
- Resultado esperado: El sistema debe descontar el monto de la cuenta y registrar el pago.
- Resultado obtenido: El pago fue registrado correctamente.

Prueba de anulación de transacción

- Objetivo: Comprobar que una transacción pueda anularse y revertirse cuando sea permitido.
- Resultado esperado: El sistema debe cambiar el estado de la transacción a ANULADA y revertir el movimiento correspondiente.
- Resultado obtenido: La transacción fue anulada correctamente.

Prueba de auditoría

- Objetivo: Verificar que las acciones importantes se registren automáticamente en la tabla de auditoría.
- Resultado esperado: Cada acción relevante debe generar un evento con usuario, módulo, operación, descripción y fecha.
- Resultado obtenido: Los eventos fueron registrados correctamente mediante triggers.

Prueba de respaldo y recuperación

- Objetivo: Comprobar que los datos respaldados puedan restaurarse después de una eliminación de prueba.
- Resultado esperado: Los datos eliminados deben recuperarse desde las tablas de respaldo.
- Resultado obtenido: La recuperación se realizó correctamente.

Prueba de consultas optimizadas

- Objetivo: Validar que las consultas principales cuenten con índices de optimización.
- Resultado esperado: Los índices deben existir y las consultas deben ejecutarse correctamente.
- Resultado obtenido: Las consultas se ejecutaron correctamente y los índices fueron creados en la base de datos.

Conclusiones

El sistema GBA CREDOMATIC permitió aplicar conceptos fundamentales de bases de datos en un escenario bancario funcional. La implementación integró usuarios, roles, clientes, cuentas, transacciones, pagos, auditoría, respaldo, recuperación y consultas optimizadas.

La base de datos en PostgreSQL permitió representar adecuadamente las relaciones entre las entidades principales del sistema, manteniendo integridad mediante llaves primarias, llaves foráneas y restricciones.

Las operaciones transaccionales implementadas demostraron la importancia de aplicar propiedades ACID en procesos bancarios, especialmente en transferencias, retiros, pagos y anulaciones.

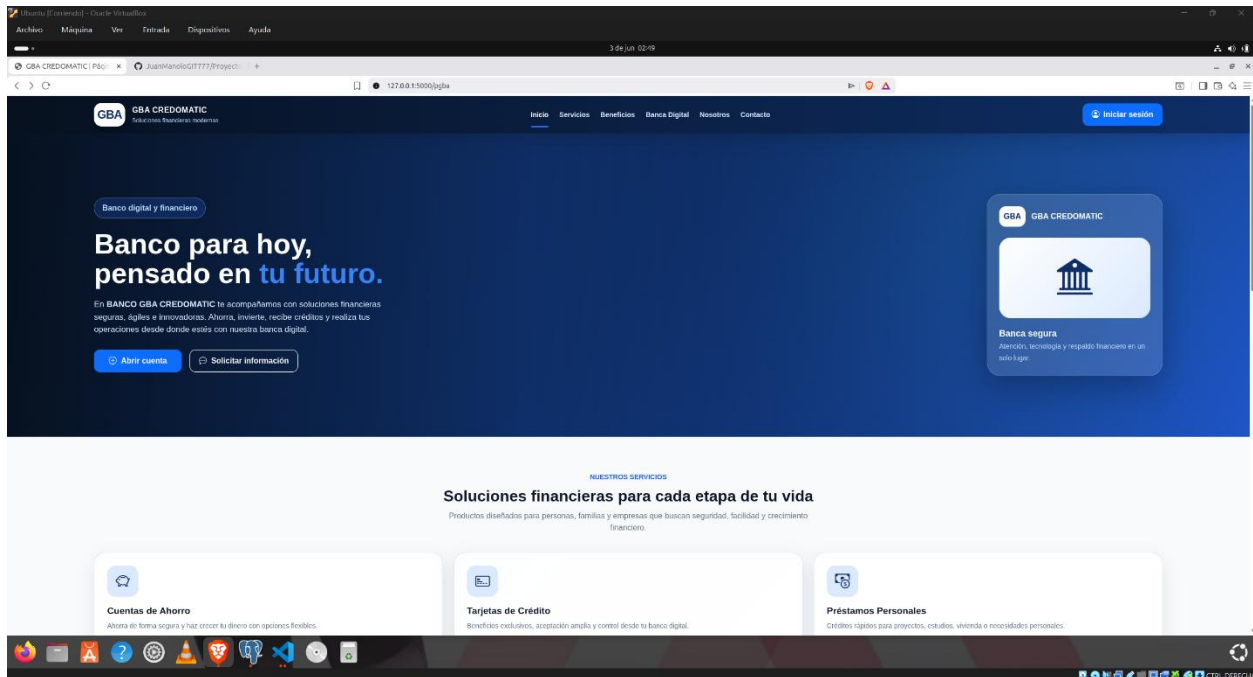
Los triggers de auditoría permitieron registrar automáticamente eventos importantes, fortaleciendo la trazabilidad y el control interno del sistema.

Los mecanismos de respaldo y recuperación permitieron demostrar cómo proteger y restaurar información crítica en caso de errores o pérdida de datos.

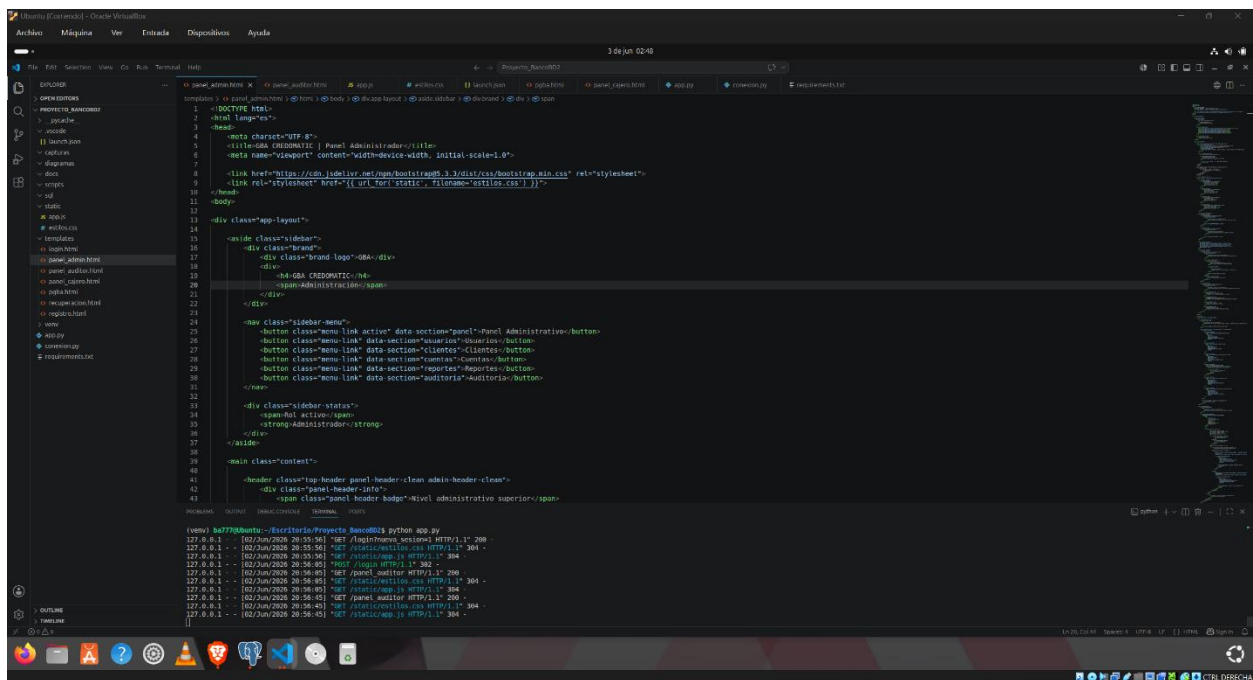
La optimización mediante índices permitió preparar la base de datos para mejorar su rendimiento cuando aumente el volumen de información.

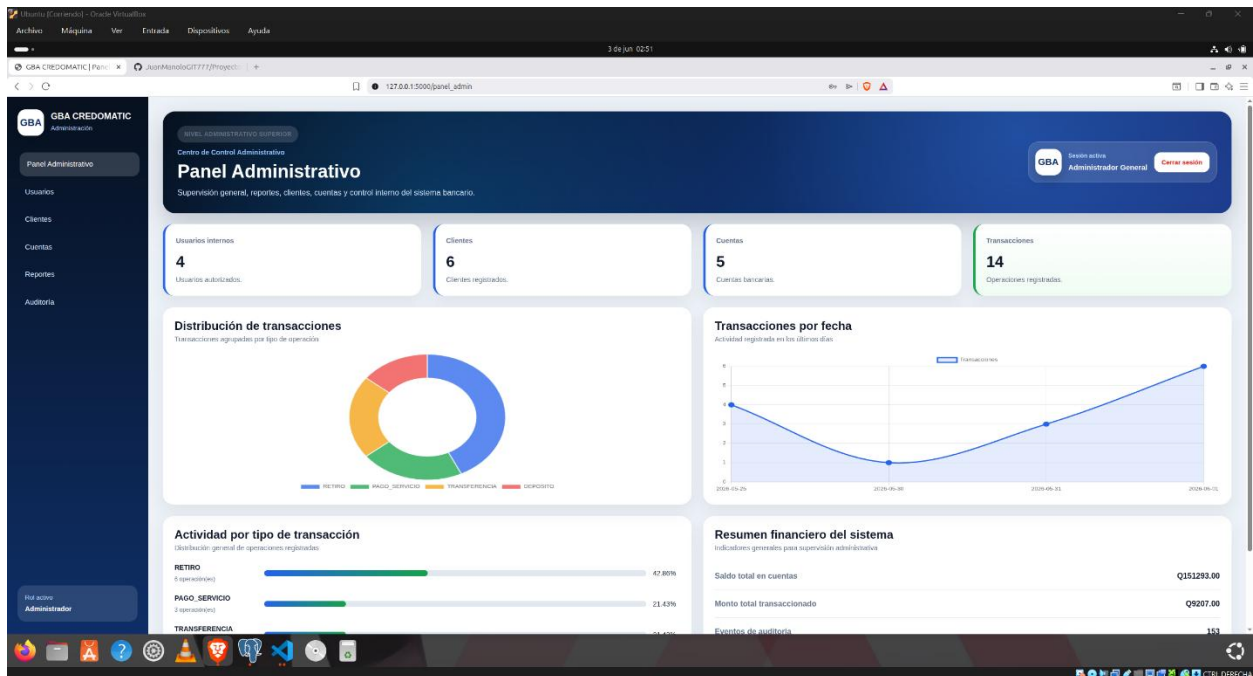
Anexos

Capturas de pantalla del Sistema GBA CREDOMATIC



Capturas del panel administrativo





Usuarios

Gestión de usuarios internos y roles autorizados.

Registrar usuario interno

Creación de usuarios administradores, cajeros o auditores del país del país.

Nombre completo: Ejemplo: Juan Pérez

Usuario: Ejemplo: jperez

Correo electrónico: usuario@correo.com

Rol: Seleccione un rol

Contraseña: Mínimo 6 caracteres

Confirmar contraseña: Repita la contraseña

Guardar usuario interno

Usuarios internos del sistema

Administración de accesos autorizados por rol.

Usuario	Nombre completo	Correo	Rol	Estado	Fecha registro	Acción
ERAdmin_banco	Erick Fernando Sanchez Monte Blanco	eradmin60@giacredomatic.com.gt	ADMINISTRADOR	ACTIVO	2026-05-30 17:29:13.154499	Contraseña activa Inactivar
auditor_banco	Auditor Interno	auditor@giacredomatic.com.gt	AUDITOR	ACTIVO	2026-05-24 23:07:15.864126	Contraseña activa Inactivar
cajero_servicio	Cajero Bancario	cajero@giacredomatic.com.gt	CAJERO	ACTIVO	2026-05-24 23:07:15.864126	Contraseña activa Inactivar
admin_banco	Administrador General	admin@giacredomatic.com.gt	ADMINISTRADOR	ACTIVO	2026-05-24 23:07:15.864126	Usuario actual

GBA CREDOMATIC Administración

Panel Administrativo

Usuarios

Cuentas

Reportes

Auditoría

GBA CREDOMATIC Administración

NIVEL ADMINISTRATIVO SUPERIOR

Centro de Control Administrativo

Cientes

Consulta administrativa de clientes registrados por el área operativa.

GBA Sesión activa: Administrador General

Cerrar sesión

Cientes registrados

Consulta administrativa de clientes creados por el área operativa.

Buscar cliente por DPI, nombre, teléfono, correo, dirección o estado...

DPI	Nombre completo	Teléfono	Correo	Dirección	Estado	Acción
79854326789057	Juan Arnoldo Escamilla Pérez	6787346	juana67@gmail.com	kilometro 200 colonia cromo alegre	ACTIVO	Consultar admin Inactivar
0985943255718	Mario Rolando Castañeda Pineda	7654793	Mario99@gmail.com	kilometro 111 avenida siempre la chona	ACTIVO	Consultar admin Inactivar
79854890273817	Luis Gustavo Martínez Arriaza	5674324	Luisg78@gmail.com	kilometro 311 avenida colonia la esperanza	ACTIVO	Consultar admin Inactivar
89564789380202	José Ramiro Álvarez Pérez	677873454	josep777@gmail.com	kilometro 888 avenida campesina 67	ACTIVO	Consultar admin Inactivar
9483763839807	Pedro Arnoldo Infante Mermucos	7847675	pedroinf@gmail.com	kilometro 777 avenida la algarra	ACTIVO	Consultar admin Inactivar
9098776790998	Juan Marcelo Pérez Palacios	1038840	juamp123@gmail.com	kilometro 7000 fd mexico	ACTIVO	Consultar admin Inactivar

Sesión activa: Administrador

GBA CREDOMATIC Administración

Panel Administrativo

Usuarios

Cuentas

Reportes

Auditoría

GBA CREDOMATIC Administración

NIVEL ADMINISTRATIVO SUPERIOR

Centro de Control Administrativo

Cuentas

Consulta administrativa de cuentas bancarias asociadas a clientes registrados.

GBA Sesión activa: Administrador General

Cerrar sesión

Cuentas bancarias

Consulta administrativa de cuentas asociadas a clientes registrados.

Buscar por número de cuenta, cliente, DPI, tipo de cuenta, saldo o estado...

Número de cuenta	Cliente	DPI	Tipo	Saldo	Estado	Fecha apertura	Acción
100001005	Juan Arnoldo Escamilla Pérez	79854326789057	EMPRESARIAL	Q99943.00	ACTIVO	2026-05-31 21:19:55.875282	Consultar admin Inactivar
100001004	Luis Gustavo Martínez Arriaza	79854890273817	AHORRO	Q7150.00	ACTIVO	2026-05-30 07:35:04.536273	Consultar admin Inactivar
100001003	José Ramiro Álvarez Pérez	89564789380202	EMPRESARIAL	Q43950.00	ACTIVO	2026-05-25 03:11:46.796358	Consultar admin Inactivar
100001002	Pedro Arnoldo Infante Mermucos	9483763839807	MONETARIA	Q0.00	ACTIVO	2026-05-25 02:32:08.833348	Consultar admin Inactivar
100001001	Juan Marcelo Pérez Palacios	9098776790998	AHORRO	Q500.00	ACTIVO	2026-05-25 01:37:43.519448	Consultar admin Inactivar

Sesión activa: Administrador

GBA

GBA CREDOMATIC

Administración

Panel Administrativo

Usuarios

Cuentas

Reportes

Auditoría

Por sesión

Administrador

NIVEL ADMINISTRATIVO SUPERIOR

Centro de Control Administrativo

Reportes

Consulta administrativa de transacciones y actividad financiera.

Sesión activa

Administrador General

Cerrar sesión

Reporte de transacciones

Supervisión administrativa de depósitos, retiros, transferencias, pagos y aruaciones

Total de transacciones:

14

Monto total:

Q9207.00

Puntos auditados:

153

Código	Tipo	Cuenta origen	Cuenta destino	Monto	Estado	Fecha	Descripción
TRX-2026060101063706406	PAGO_SERVICIO	5	N/A	Q77.00	COMPLETADO	2026-06-01 01:06:57.070596	Pago de servicio con referencia 8705430087051.
TRX-2026060101054698622	RETIRO	2	N/A	Q800.00	COMPLETADO	2026-06-01 01:05:46.998020	Retiro realizado en ventanilla.
TRX-2026060101053588663	RETIRO	5	N/A	Q800.00	COMPLETADO	2026-06-01 01:05:35.896397	Retiro realizado en ventanilla.
TRX-20260601010457374075	TRANSFERENCIA	5	2	Q100.00	COMPLETADO	2026-06-01 01:04:57.364541	Transferencia realizada en ventanilla.
TRX-20260601010436488137	DEPOSITO	N/A	4	Q300.00	COMPLETADO	2026-06-01 01:04:35.478462	Depósito realizado en ventanilla.
TRX-20260601010350943472	DEPOSITO	N/A	4	Q800.00	COMPLETADO	2026-06-01 01:03:50.954652	Depósito realizado en ventanilla.
TRX-20260531232432979225	RETIRO	5	N/A	Q500.00	COMPLETADO	2026-05-31 23:24:32.988182	Retiro realizado en ventanilla.
TRX-2026053121244635645	PAGO_SERVICIO	5	N/A	Q80.00	COMPLETADO	2026-05-31 21:21:46.848328	Pago de servicio con referencia 6754576970543.
TRX-2026053121203201866	TRANSFERENCIA	3	5	Q1000.00	COMPLETADO	2026-05-31 21:20:30.212437	Transferencia realizada en ventanilla.
TRX-20260530147477777381	TRANSFERENCIA	3	2	Q390.00	COMPLETADO	2026-05-30 14:24:27.727870	Transferencia realizada en ventanilla.
TRX-20260529051234088755	RETIRO	3	N/A	Q5000.00	COMPLETADO	2026-05-29 05:12:14.108992	Retiro realizado en ventanilla.
TRX-20260525023330140531	PAGO_SERVICIO	1	N/A	Q250.00	COMPLETADO	2026-05-25 02:33:30.157597	Pago de servicio con referencia 4354565757057.
TRX-20260525023241739503	RETIRO	2	N/A	Q50.00	COMPLETADO	2026-05-25 02:32:41.747647	Retiro realizado en ventanilla.
TRX-20260525013006430225	RETIRO	1	N/A	Q700.00	COMPLETADO	2026-05-25 01:38:06.439945	Retiro realizado en ventanilla.

GBA

GBA CREDOMATIC

Administración

Panel Administrativo

Usuarios

Cuentas

Reportes

Auditoría

Por sesión

Administrador

NIVEL ADMINISTRATIVO SUPERIOR

Centro de Control Administrativo

Panel Administrativo

Supervisión general, reportes, clientes, cuentas y control interno del sistema bancario.

Sesión activa

Administrador General

Cerrar sesión

Usuarios internos:

4

Usuarios autorizados:

Clientes:

6

Clientes registrados:

Cuentas:

5

Cuentas bancarias:

Transacciones:

14

Operaciones registradas:

Distribución de transacciones

Transacciones agrupadas por tipo de operación

Transacciones por fecha

Actividad registrada en los últimos días

Actividad por tipo de transacción

Visualización general de operaciones registradas

RETIRO

5 operaciones(s)

47.80%

PAGO_SERVICIO

3 operaciones(s)

21.43%

TRANSFERENCIA

6 operaciones(s)

42.76%

Resumen financiero del sistema

Indicadores generales para supervisión administrativa

Saldo total en cuentas

Q151293.00

Monto total transaccionado

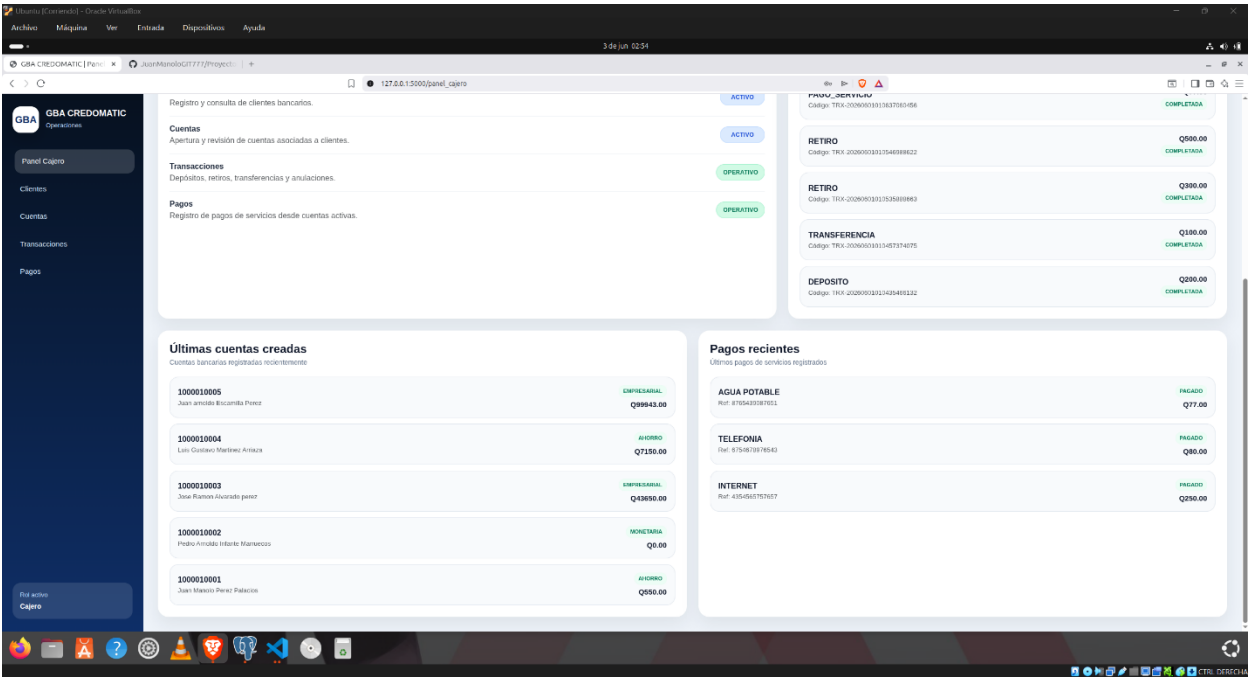
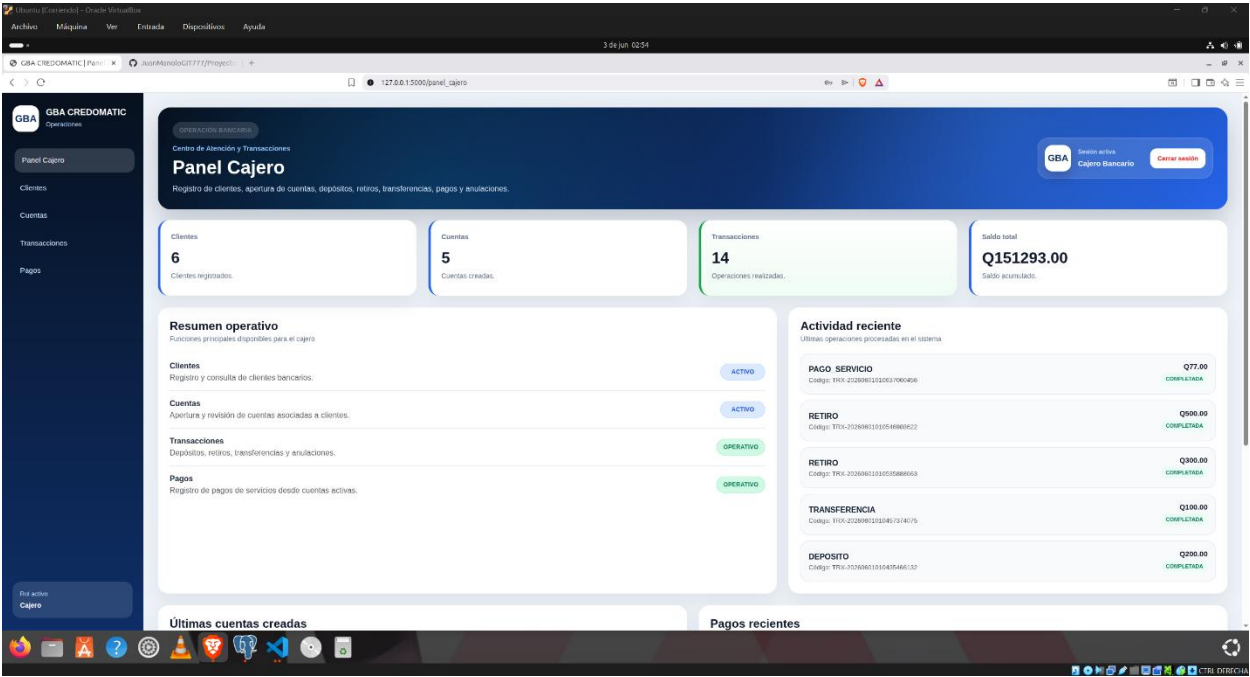
Q9207.00

Puntos de auditoría

153

171

Capturas del módulo de clientes



Ubuntu [convenio] - Oracle VM VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

3 de jun. 02:51

GBA CREDOMATIC [Pantalla] - Juan Manuel Gutierrez [Proyecto] - 127.0.0.1:5000/panel_cajero

GBA CREDOMATIC Operaciones

Panel Cajero

Cajeros

Cuentas

Transacciones

Pagos

Por activar

Cajero

GBA CREDOMATIC Operaciones

Centro de Atención y Transacciones

Cajero Bancario

Cerrar sesión

Cientes

Registro y consulta de clientes bancarios.

Registrar cliente

Ingreso de datos básicos del cliente bancario

DPI: Nombres: Apellidos:

Teléfono: Correo: Dirección:

[Guardar cliente](#)

Cientes registrados

Lista de clientes bancarios

DPI	Nombre completo	Teléfono	Correo	Dirección	Estado
78954321098765	Juan Amado Escamilla Perez	6787345	juan67@gmail.com	Kilometro 200 coloris como alegre	ACTIVO
09856432109876	Mario Rolando Castañeda Pineda	7654321	Mario98@gmail.com	Kilometro 111 avenida siempre la chona	ACTIVO
70954090273817	Luis Gustavo Martinez Ariza	5674321	Luis78@gmail.com	Kilometro 311 avenida coloris la esperanza	ACTIVO
89564709381202	José Barrón Alvarado Perez	67875454	jpmp777@gmail.com	Kilometro 088 avenida siempre la esperanza 67	ACTIVO
94857063839202	Pedro Amado Infante Mantueros	7847875	pedroam6@gmail.com	Kilometro 777 avenida la abeja	ACTIVO
9098778750098	Juan Manuel Perez Palacios	1098843	juanp123@gmail.com	Kilometro 7000 ki meaco	ACTIVO

Ubuntu [convenio] - Oracle VM VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

3 de jun. 02:51

GBA CREDOMATIC [Pantalla] - Juan Manuel Gutierrez [Proyecto] - 127.0.0.1:5000/panel_cajero

GBA CREDOMATIC Operaciones

Panel Cajero

Cajeros

Cuentas

Transacciones

Pagos

Por activar

Cajero

GBA CREDOMATIC Operaciones

Centro de Atención y Transacciones

Cajero Bancario

Cerrar sesión

Cuentas

Apertura y consulta de cuentas bancarias.

Apertura de cuenta

Creación de una cuenta bancaria asociada a un cliente

Cliente: Tipo de cuenta: Saldo inicial:

[Crear cuenta](#)

Cuentas registradas

Lista de cuentas bancarias activas o registradas

Número de cuenta	Cliente	Tipo	Saldo	Estado
1000010005	Juan Amado Escamilla Perez	EMPRESARIAL	Q99943.00	ACTIVO
1000010004	Luis Gustavo Martinez Ariza	AHORRO	Q7130.00	ACTIVO
1000010003	José Barrón Alvarado Perez	EMPRESARIAL	Q43850.00	ACTIVO
1000010002	Pedro Amado Infante Mantueros	MONETARIA	Q0.00	ACTIVO
1000010001	Juan Manuel Perez Palacios	AHORRO	Q560.00	ACTIVO

GBA CREDOMATIC [Proyecto] - Operación Bancaria

Panel Cajero

Cuentas

Transacciones

Pagos

Últimas transacciones

Historial reciente de movimientos registrados:

Código	Tipo	Origen	Destino	Monto	Estado	Fecha
TRX-202606010037060456	PAGO, SERVICIO	5	N/A	Q77.00	COMPLETADA	2026-06-01 01:05:37.070996
TRX-2026060100354808022	RETIRO	2	N/A	Q200.00	COMPLETADA	2026-06-01 01:05:46.998030
TRX-2026060100355809663	RETIRO	5	N/A	Q200.00	COMPLETADA	2026-06-01 01:05:35.996387
TRX-2026060100457374075	TRANSFERENCIA	5	2	Q100.00	COMPLETADA	2026-06-01 01:04:57.384041
TRX-2026060100432646132	DEPOSITO	N/A	4	Q200.00	COMPLETADA	2026-06-01 01:04:35.419482

GBA CREDOMATIC [Proyecto] - Operación Bancaria

Panel Cajero

Cuentas

Transacciones

Pagos

Pagos de servicio

Pago de servicios asociados a una cuenta bancaria:

Cuenta	Servicio	Referencia	Monto
Selección cuenta	Selección servicio		

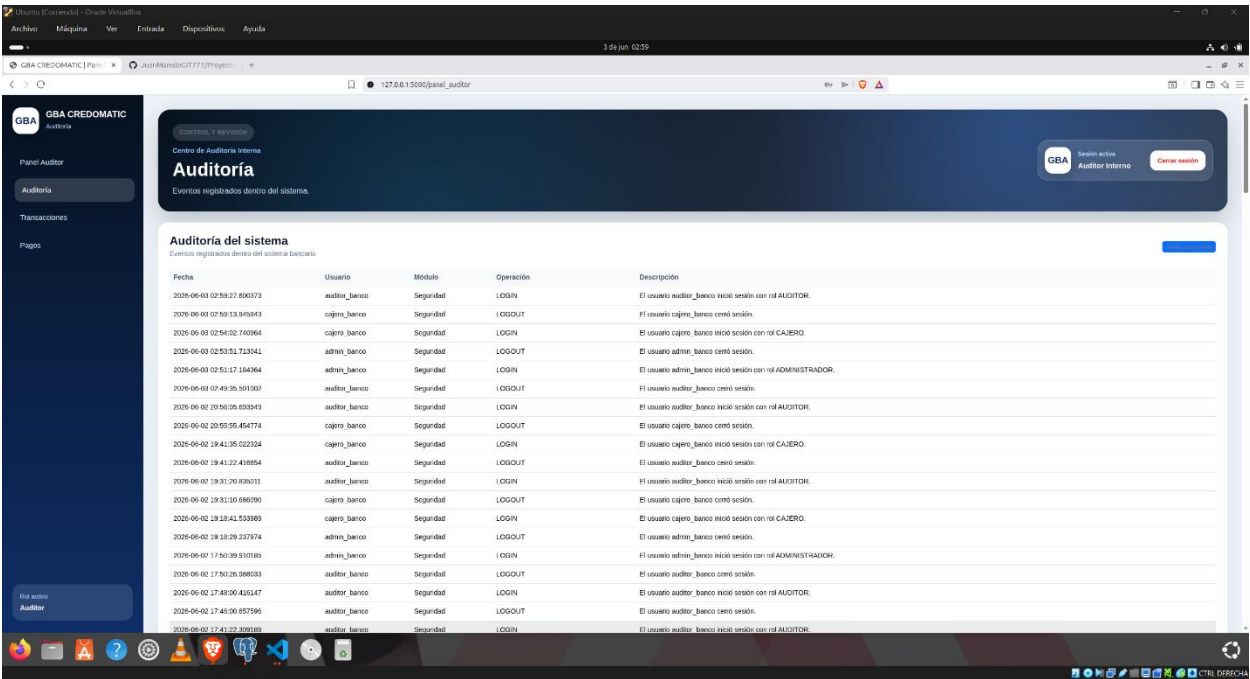
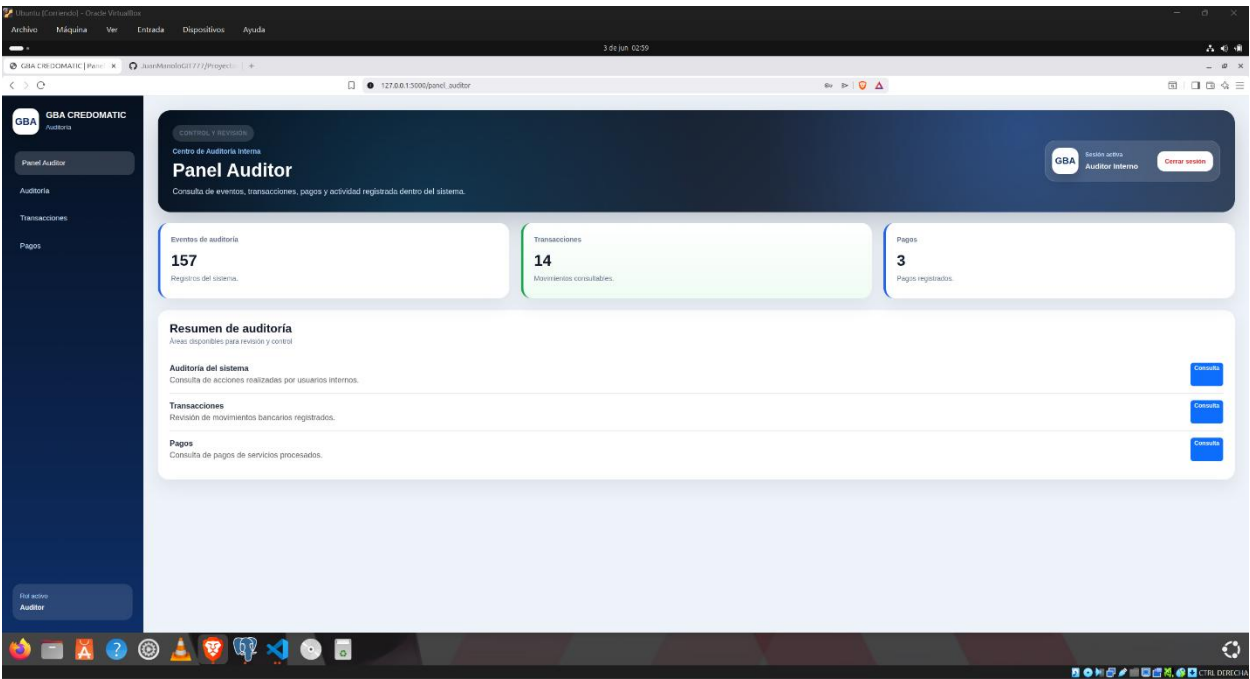
Registrar pago

Pagos registrados

Historial reciente de pagos de servicios:

Código	Cuenta	Servicio	Referencia	Monto	Estado	Fecha
PAG-202606010037060436	1800010005	AGUA POTABLE	6765439087951	Q77.00	PAGADO	2026-06-01 01:06:37.070996
PAG-2026060121214803629	1800010005	TELEFONIA	6754678976543	Q80.00	PAGADO	2026-05-31 21:21:48.848329
PAG-2026060150073303148013	1800010001	INTERNET	4304965737857	Q250.00	PAGADO	2026-05-25 07:33:30.157997

Capturas del panel auditor



GBA CREDOMATIC Auditor

Panel Auditor

Auditoría

Transacciones

Pagos

GBA CREDOMATIC Auditor

Centro de Auditoría Interna

Transacciones

Consulta de movimientos bancarios registrados.

Transacciones registradas

Consulta de movimientos bancarios procesados.

Código	Tipo	Cuenta origen	Cuenta destino	Monto	Estado	Fecha	Descripción
TRX-202608010108706458	PAGO SERVICIO	5	N/A	Q77.00	COMPLETADA	2026-08-01 01:06:37.070656	Pago de servicio con referencia 6765439087851.
TRX-20260801010546886072	RETIRO	7	N/A	Q200.00	COMPLETADA	2026-08-01 01:05:46.398020	Retiro realizado en ventanilla.
TRX-20260801010538886653	RETIRO	5	N/A	Q300.00	COMPLETADA	2026-08-01 01:05:39.896307	Retiro realizado en ventanilla.
TRX-20260801010457374075	TRANSFERENCIA	5	2	Q100.00	COMPLETADA	2026-08-01 01:04:57.384041	Transferencia realizada en ventanilla.
TRX-20260801010455465132	DEPOSITO	N/A	4	Q200.00	COMPLETADA	2026-08-01 01:04:55.479402	Deposito realizado en ventanilla.
TRX-20260801010350543472	DEPOSITO	N/A	4	Q500.00	COMPLETADA	2026-08-01 01:03:50.954652	Deposito realizado en ventanilla.
TRX-2026080101032432970276	RETIRO	5	N/A	Q100.00	COMPLETADA	2026-08-01 01:03:24.32388182	Retiro realizado en ventanilla.
TRX-202608010102248839545	PAGO SERVICIO	5	N/A	Q85.00	COMPLETADA	2026-08-01 01:02:48.848329	Pago de servicio con referencia 6754378970543.
TRX-202608010102030201666	TRANSFERENCIA	3	5	Q1000.00	COMPLETADA	2026-08-01 01:02:03.20212437	Transferencia realizada en ventanilla.
TRX-2026080101014427227387	TRANSFERENCIA	3	2	Q350.00	COMPLETADA	2026-08-01 01:01:44.27237970	Transferencia realizada en ventanilla.
TRX-2026080101013714089705	RETIRO	3	N/A	Q2000.00	COMPLETADA	2026-08-01 01:01:37.14108897	Retiro realizado en ventanilla.
TRX-20260801010133303145831	PAGO SERVICIO	1	N/A	Q200.00	COMPLETADA	2026-08-01 01:01:33.30157097	Pago de servicio con referencia 4304560787807.
TRX-20260801010122411735503	RETIRO	2	N/A	Q60.00	COMPLETADA	2026-08-01 01:01:22.411747847	Retiro realizado en ventanilla.
TRX-20260801010106430225	RETIRO	1	N/A	Q200.00	COMPLETADA	2026-08-01 01:01:06.4309945	Retiro realizado en ventanilla.

Est active Auditor

GBA CREDOMATIC Auditor

Panel Auditor

Auditoría

Transacciones

Pagos

GBA CREDOMATIC Auditor

Centro de Auditoría Interna

Pagos

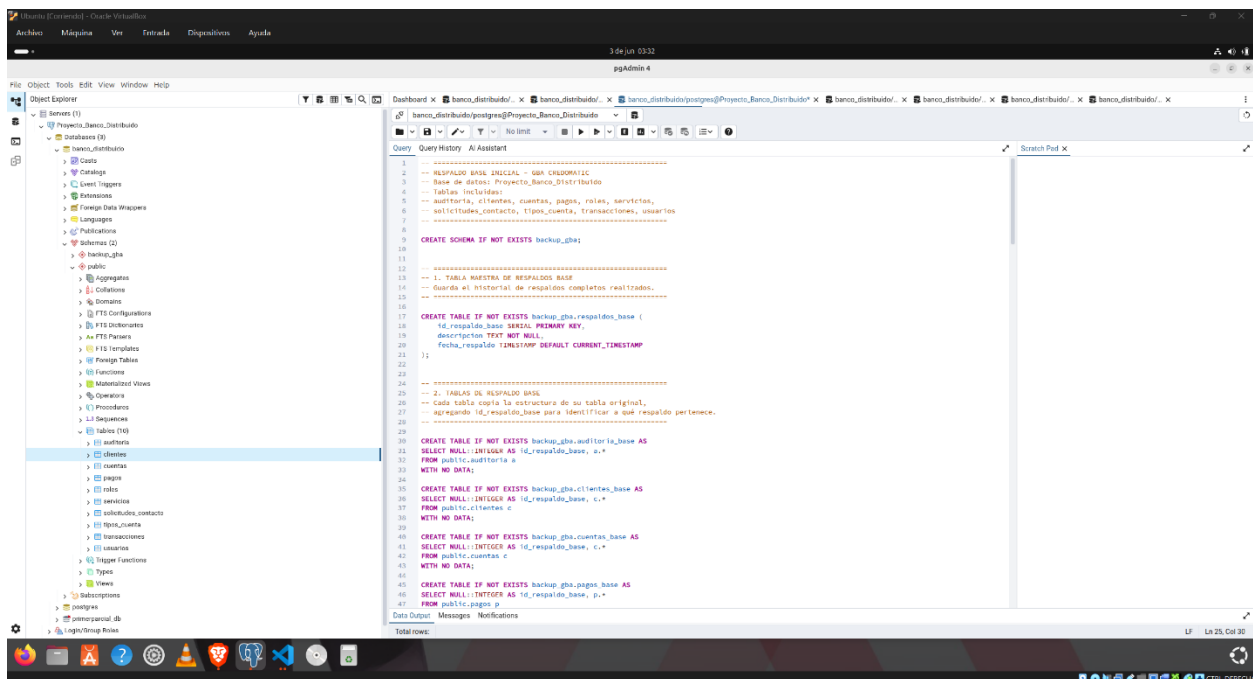
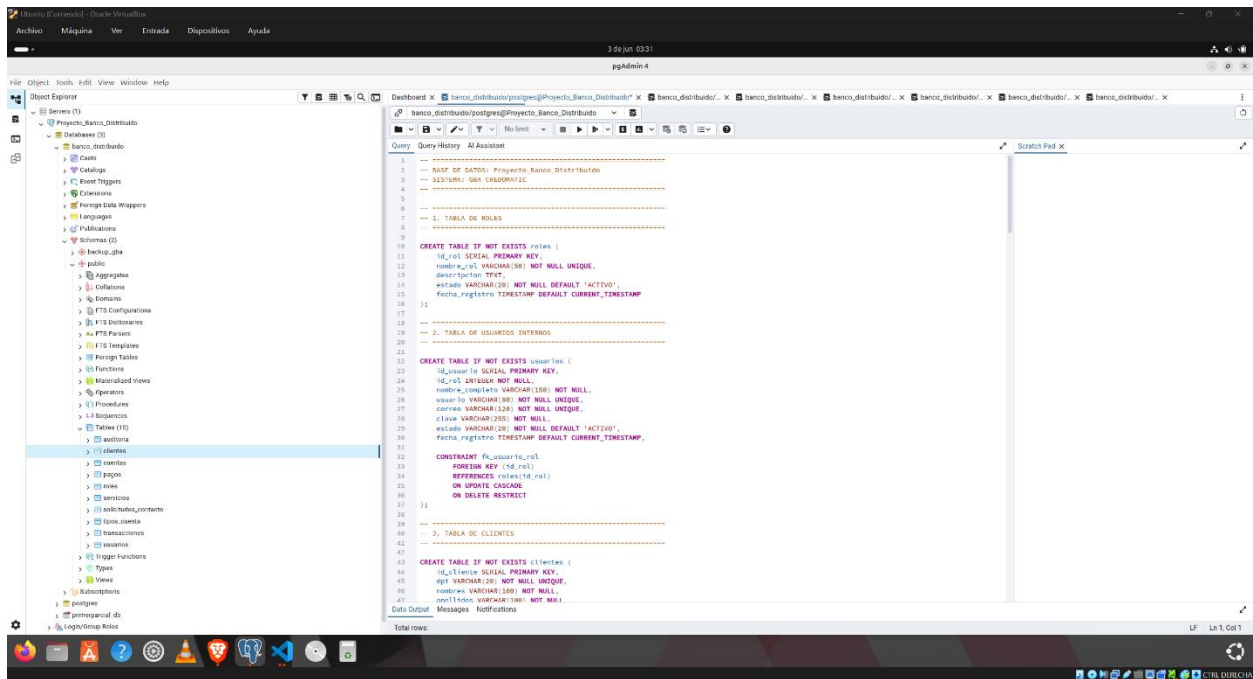
Consulta de pagos de servicios registrados.

Pagos registrados

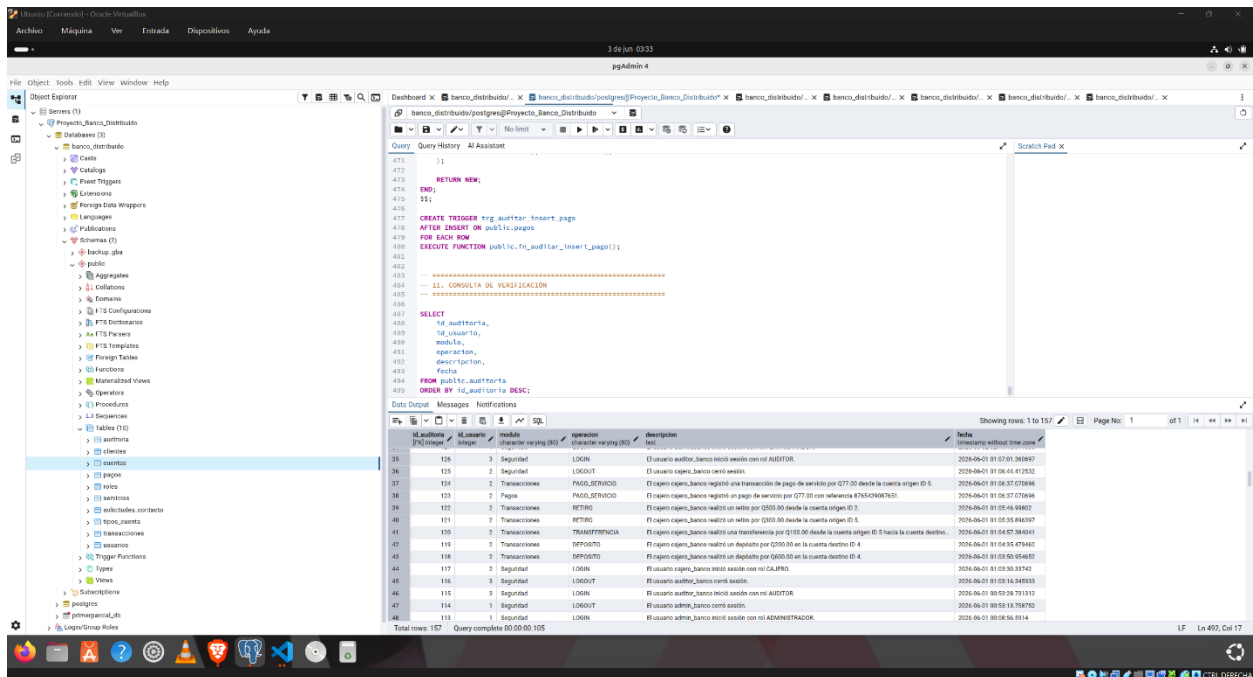
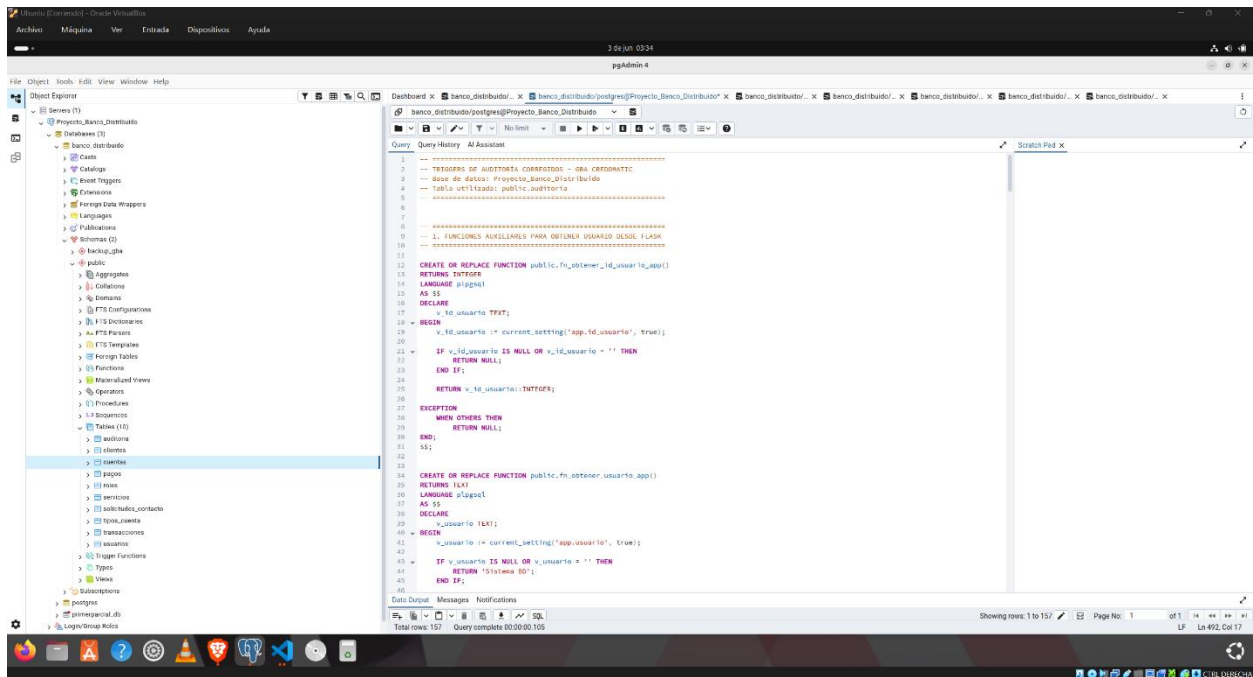
Consulta de pagos de servicios realizados.

Código	Cuenta	Servicio	Referencia	Monto	Estado	Fecha
PAG-202608010108706436	1000010005	AGUA POTABLE	6765439087851	Q77.00	PAGADO	2026-08-01 01:06:37.070656
PAG-2026080101054688629	1000010005	TEL FICHA	6754678970543	Q80.00	PAGADO	2026-08-01 01:05:46.398020
PAG-20260801010350543472	1000010001	INTERNET	4304560787807	Q200.00	PAGADO	2026-08-01 01:03:50.954652

Est active Auditor



Capturas de triggers ejecutados



The screenshot shows a PostgreSQL database interface. On the left, the 'Object Explorer' pane displays the database structure for 'banco_distribuido'. It includes a 'Schemas' section with 'public' and 'backup_gba', and a 'Tables' section with various tables like 'auditoria', 'clientes', 'cuentas', 'pagos', 'servicios', 'usuarios', 'transacciones', and 'transacciones_usuario'. The 'Tables' section is expanded, showing a list of tables including 'auditoria', 'clientes', 'cuentas', 'pagos', 'servicios', 'usuarios', 'transacciones', and 'transacciones_usuario'. The 'cuentas' table is selected.

On the right, the 'Query Editor' pane shows a SQL script for creating a backup schema and tables. The script includes comments in Spanish and SQL commands for creating a schema, a table, and several other tables with specific constraints and data types.

```

1  --
2  -- RESPALDO BASE INICIAL - GBA CLOUDMATIC
3  -- Base de datos Proyecto_Banco_Distribuido
4  -- Tablas incluidas:
5  -- auditoria, clientes, cuentas, pagos, roles, servicios,
6  -- solicitudes, contactos, tipos_cuenta, transacciones, usuarios
7  --
8  --
9  CREATE SCHEMA IF NOT EXISTS backup_gba;
10
11
12  --
13  -- 1. TABLA MAESTRO DE RESPALDOS BASE
14  -- Guarda el historial de respaldos completos realizados.
15  --
16  CREATE TABLE IF NOT EXISTS backup_gba.respaldo_base (
17      id_respaldo_base SERIAL PRIMARY KEY,
18      descripcion TEXT NOT NULL,
19      fecha_respaldo TIMESTAMP DEFAULT CURRENT_TIMESTAMP
20  );
21
22
23  --
24  -- 2. TABLAS DE RESPALDO BASE
25  -- Crea tabla copia la estructura de su tabla original,
26  -- agregando id_respaldo_base para identificar a qué respaldo pertenece.
27  --
28  --
29  CREATE TABLE IF NOT EXISTS backup_gba.auditoria_base AS
30  SELECT NULL::INTEGER AS id_respaldo_base, s.*
31  FROM public.auditoria a
32  WITH NO DATA;
33
34  CREATE TABLE IF NOT EXISTS backup_gba.clientes_base AS
35  SELECT NULL::INTEGER AS id_respaldo_base, c.*
36  FROM public.clientes c
37  WITH NO DATA;
38
39  CREATE TABLE IF NOT EXISTS backup_gba.cuentas_base AS
40  SELECT NULL::INTEGER AS id_respaldo_base, c.*
41  FROM public.cuentas c
42  WITH NO DATA;
43
44  CREATE TABLE IF NOT EXISTS backup_gba.pagos_base AS
45  SELECT NULL::INTEGER AS id_respaldo_base, p.*
46  FROM public.pagos p
47  WITH NO DATA;
48
49  Data Output Messages Notifications
50
51  Total rows

```

179

The screenshot shows a PostgreSQL database interface. On the left, the 'Object Explorer' pane displays the database structure, including 'Databases (2)', 'Schemas (1)', and 'Tables (10)'. The 'cuotas' table is selected under the 'public' schema. The main window shows a SQL query in the 'Query' editor, which is a complex query involving a subquery and a main query. The query is as follows:

```

-- CONSULTA 1: HISTORIAL DE MOVIMIENTOS POR CUENTA
-- Sirve para ver depósitos, retiradas y pagos
-- donde pertenezca una cuenta.
-- Cambiar el número de cuenta por uno ir.
-----
-- consulta normal
SELECT
    t.codigo_transaccion,
    t.fecha_transaccion,
    co.numero_cuenta AS cuenta_origen,
    cd.numero_cuenta AS cuenta_destino,
    t.monto,
    t.estado,
    t.fecha_transaccion
FROM public.transacciones t
LEFT JOIN public.cuentas co ON t.id_cuenta_origen = co.id_cuenta
LEFT JOIN public.cuentas cd ON t.id_cuenta_destino = cd.id_cuenta
WHERE co.numero_cuenta = '1000010001'
OR cd.numero_cuenta = '1000010001'
ORDER BY t.fecha_transaccion DESC;

```

Below the query editor, the 'Data Output' pane shows the results of the query. The results are displayed in a table with the following columns: 'codigo_transaccion', 'numero_cuenta_origen', 'numero_cuenta_destino', 'monto', 'estado', and 'fecha_transaccion'. The table contains two rows of data.

codigo_transaccion	numero_cuenta_origen	numero_cuenta_destino	monto	estado	fecha_transaccion
TRX 20200920071854042...	PAGO_SERVICO	1000010001	250.00	COMPLETADA	2020-09-20 02:59:30.179797
TRX 20200920071854042...	RETIRO	1000010001	200.00	COMPLETADA	2020-09-20 03:38:36.407945

The status bar at the bottom indicates 'Total rows: 2' and 'Query completes 00:00:01.150'.

180

Enlace al repositorio del proyecto

https://github.com/JuanManoloGIT777/ProyectoBD2_bancodistribuido_GBA.git